

Instituto Politécnico Nacional

Escuela Superior de Cómputo

ESCOM

Trabajo Terminal I

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

2027-A178

Ingeniería en Sistemas Computacionales

Presentan

Sánchez Catarino Edgar
Linares Villegas Gustavo

Director

Dr. en C. Enrique Escalante Notario



2026

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA

No. de TT: 2027-A178 12 de mayo de 2026

Documento técnico

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

2027-A178

Presentan

Sánchez Catarino Edgar

Linares Villegas Gustavo

Director

Dr. en C. Enrique Escalante Notario

RESUMEN

Este Trabajo Terminal propone diseñar e implementar un General Game Player (GGP) para juegos discretos, deterministas, por turnos y de información perfecta. La idea es separar las reglas de cada juego del motor de decisión del agente mediante un lenguaje de descripción externo. A partir de ese lenguaje, el sistema interpretará la estructura del juego, construirá una representación interna, generará los movimientos legales, aplicará transiciones, reconocerá los estados terminales y elegirá acciones con algoritmos generales de búsqueda. El proyecto se validará con el Gato, como caso funcional completo, y con el Ajedrez, como caso de mayor complejidad cuyo diseño experimental aún debe definirse, para observar qué tanto generaliza el enfoque sin modificar el núcleo algorítmico del agente.

Palabras clave: General Game Playing, lenguaje de descripción de juegos, inteligencia artificial, juegos de información perfecta, árboles de búsqueda, Monte Carlo Tree Search.

Advertencia

"Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos sujetos a derechos de propiedad. Por lo tanto, su uso queda restringido a las aplicaciones que explícitamente se convengan."

Cualquier aplicación no convenida exime a la Escuela Superior de Cómputo de responsabilidad técnica y podrá dar lugar a las consecuencias legales que, para tal efecto, correspondan.

Para obtener información adicional sobre este reporte técnico, podrá dirigirse a:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional.

Índice general

Advertencia II

Glosario IX

Abreviaturas XII

1 Introducción **1**
1.1 Contexto y motivación 1
1.2 Planteamiento del problema 2
1.3 Justificación 3
1.4 Objetivos 4
1.4.1 Objetivo general 4
1.4.2 Objetivos específicos 4
1.5 Alcance y limitaciones 4
1.6 Metodología y planeación del desarrollo 5
1.6.1 Enfoque iterativo e incremental 5
1.6.2 Etapas de desarrollo 5
1.6.3 Productos esperados 6
1.6.4 Matriz de seguimiento y evidencias propuestas 6
1.6.5 Diseño de la evaluación: métricas y muestra 7
1.6.6 Distribución del trabajo y calendario 9

2 Estado del arte **11**
2.1 Evolución de los sistemas de juego en inteligencia artificial 11
2.2 Surgimiento y fundamentos de General Game Playing 12
2.3 Generaciones y agentes emblemáticos de General Game Playing 12
2.3.1 Primera generación: Análisis estático de reglas y Minimax heurístico . . 12
2.3.2 Segunda generación: La revolución de Monte Carlo Tree Search 13
2.3.3 Tercera generación: PropNets optimizadas y heurísticas de simulación . 14
2.4 Plataformas y lenguajes modernos de descripción de juegos 15
2.4.1 Extensiones de GDL e información imperfecta 15
2.4.2 El sistema Ludii 15
2.4.3 Regular Boardgames 15
2.4.4 Entornos generales para aprendizaje: OpenSpiel 16
2.5 Tendencias contemporáneas en aprendizaje profundo y modelos unificados . . . 16
2.6 Síntesis comparativa y posicionamiento del Trabajo Terminal 16

3 Marco Teórico **18**
3.1 Inteligencia artificial en juegos . . . 18
3.2 Juegos de información perfecta . . . 18
3.2.1 Espacio de observaciones y función de observación (O_i) . . . 19
3.2.2 Relación de indistinguibilidad ($\sim_i$) . . . 19
3.2.3 La indistinguibilidad como relación de equivalencia . . . 20
3.2.4 Conjuntos de información y clases de observación . . . 20
3.2.5 Ejemplos comparativos . . . 21
3.2.6 Implicación para el modelo formal Γ . . . 22
3.3 General Game Playing . . . 23
3.4 Modelo formal de juego . . . 23
3.4.1 Conjunto de roles (R) . . . 24
3.4.2 Espacio de estados (S) y semántica de fluents . . . 24
3.4.3 Estado inicial (s_0) . . . 26
3.4.4 Acciones individuales (A_i) y espacio conjunto de acciones ($\mathbf{A}$) . . . 26
3.4.5 Función de legalidad (L_i) . . . 26
3.4.6 Función de transición dinámica (T) . . . 26
3.4.7 Conjunto de estados terminales (F) . . . 27
3.4.8 Función de utilidad o meta (G_i) . . . 27
3.4.9 Axiomas de admisibilidad y juegos bien formados . . . 27
3.4.10 Distancia en el grafo de estados y estimación hacia configuraciones terminales . . . 28
3.4.11 Modelado síncrono de juegos por turnos alternados . . . 30
3.4.12 Interfaz computacional para el motor de decisión . . . 30
3.5 Lenguaje de descripción de juegos (GDL) . . . 30
3.5.1 Características fundamentales de GDL . . . 31
3.5.2 Concepto y estructura de un predicado lógico . . . 31
3.5.3 Predicados reservados del sistema en GDL . . . 32
3.5.4 Ejemplificación mediante código de especificación: El juego del Gato . . . 32
3.6 GDL, Ludii y lenguajes de dominio específico . . . 34
3.7 Intérprete y representación interna . . . 36
3.8 Árboles de juego y explosión combinatoria . . . 36
3.9 Minimax . . . 37
3.10 Poda alfa-beta . . . 38
3.11 Funciones heurísticas generales . . . 38
3.12 Monte Carlo Tree Search . . . 39
3.13 UCT . . . 40
3.14 Aprendizaje por refuerzo y aprendizaje profundo . . . 40
3.15 AlphaZero como antecedente moderno . . . 41
3.16 Síntesis del marco teórico . . . 41

4 Análisis **43**

5 Diseño **44**
5.1 Arquitectura conceptual propuesta . . . 44

5.2 Prototipos de interfaz aprobados 46

6 Desarrollo **52**

7 Pruebas **53**

8 Conclusiones y trabajo a futuro **54**

Referencias **55**

Anexos **59**
Cronogramas 59
Sánchez Catarino Edgar 59
Linares Villegas Gustavo 60

Índice de figuras

1.1 Comparación estructural entre un motor especializado tradicional y la arquitectura desacoplada de un General Game Player. 3

2.1 Comparación esquemática entre la inferencia lógica interpretada recursivamente y la evaluación acelerada por propagación de señales en redes proposicionales (*PropNets*). 13
2.2 Organización expositiva de antecedentes y generaciones algorítmicas en *General Game Playing* (GGP). 14

3.1 Clases de estados compatibles con una observación. En (a), dos estados producen la misma observación y pertenecen a una clase no unitaria; en (b), la observación inyectiva identifica cada estado. La figura ilustra observación de estados, no conjuntos de información sobre historias. 22
3.2 Árbol de juego adversarial con poda alfa-beta. Tras evaluarse el nodo con valor 2, el nodo MIN asegura un valor a lo sumo de 2, el cual es estrictamente inferior al valor $\alpha = 3$ ya garantizado por MAX en la rama izquierda; en consecuencia, se interrumpe la exploración del resto de los sucesores. 38
3.3 Ciclo iterativo de cuatro fases en Monte Carlo Tree Search (MCTS): Selección de nodos mediante la cota UCT, Expansión de nuevas acciones legales, Simulación estocástica (*rollout*) hasta un estado terminal y Retropropagación de recompensas hacia la raíz. 39
3.4 Flujo conceptual de ejecución y arquitectura desacoplada del General Game Player. El flujo detallado se presenta en la Figura 5.1. 42

5.1 Arquitectura propuesta: carga de reglas, consultas de búsqueda y actualización de la partida. Las dos filas principales forman un único flujo; las entradas, salidas y condiciones de terminación se muestran alrededor del núcleo. 45
5.2 Prototipo aprobado de carga y revisión de una descripción externa de Gato. La validación y el contenido del archivo son ilustrativos; la sintaxis del lenguaje permanece por definir. 47
5.3 Prototipo aprobado de una partida de Gato entre agentes. El tablero, historial, acciones legales y próxima decisión representan un estado ilustrativo. 48
5.4 Prototipo aprobado de una partida de Ajedrez entre agentes. La posición y las acciones son ilustrativas. 49
5.5 Prototipo aprobado de preparación de una evaluación táctica de primera jugada en Ajedrez. Las cantidades, el presupuesto y el estado de ejecución son ilustrativos y no describen un experimento realizado. 50

5.6 Prototipo aprobado del reporte de resultados de una evaluación. Las métricas y el detalle por caso muestran datos simulados, sin atribuir una ejecución real ni desempeño al agente. 51

Índice de tablas

1.1 Matriz propuesta de seguimiento del desarrollo. 6
1.2 Métricas y procedimientos propuestos de evaluación. 8

2.1 Comparación de paradigmas y sistemas representativos en juegos e inteligencia artificial. 17

3.1 Predicados reservados de GDL y componente de Γ que describen. 32

Glosario

Término	Definición
Acción legal	Movimiento que un jugador puede hacer en un estado concreto del juego, según las reglas de la descripción formal del dominio.
Acción nula (*noop*)	Acción pasiva (*no-operation*) que ejecutan los jugadores que no tienen el turno. Sirve para representar la alternancia de turnos dentro de un modelo formal síncrono y con varios participantes.
Agente inteligente	Sistema que percibe un entorno, razona sobre sus posibles acciones y elige una de acuerdo con un objetivo, una recompensa o un criterio de utilidad.
Árbol de juego	Grafo acíclico con las secuencias posibles de estados y acciones de una partida. Cada nodo es una configuración y cada arista, una acción conjunta que lleva a un nuevo estado.
Asunción de mundo cerrado (*CWA*)	Principio de la programación lógica según el cual todo hecho o *fluent* que no pueda deducirse como verdadero en el estado actual se considera falso.
Conjunto de información de Kuhn	Conjunto de nodos de decisión o historias que un jugador no distingue con la información recibida hasta ese momento. No equivale automáticamente a una clase de estados con igual observación actual.
Criterio de selección UCT	Regla de selección de Monte Carlo Tree Search basada en la cota superior de confianza UCB1. Busca equilibrar la explotación de las jugadas que han funcionado con la exploración de las ramas poco visitadas.
Descripción de juego	Especificación externa y declarativa que define los roles, el estado inicial, las acciones legales, las transiciones, los estados terminales y las recompensas, sin programar esas reglas dentro del motor del agente.
Distancia dirigida entre estados	Longitud mínima de una trayectoria legal dirigida entre estados. Vale cero de un estado a sí mismo e infinito si no hay trayectoria; no es una métrica ordinaria porque puede ser asimétrica y no finita.
Estado	Configuración completa del juego en un instante discreto. Formalmente, es el conjunto finito de *fluents* atómicos verdaderos bajo la asunción de mundo cerrado.
Estado terminal	Configuración en la que la partida termina y se asigna una recompensa numérica a cada participante.
Explosión combinatoria	Crecimiento exponencial del número de configuraciones y de nodos del árbol de búsqueda a medida que se profundiza, provocado por el factor de ramificación del dominio.

Continúa en la siguiente página...

Término	**Definición**
Factor de ramificación	Número promedio de acciones legales que salen de un nodo del árbol de búsqueda de un juego.
Fluent	Proposición lógica atómica cuyo valor de verdad cambia con el tiempo o con el estado. Representa hechos del entorno que pueden aparecer, mantenerse o desaparecer tras cada transición.
Función de observación	Función que asigna a cada estado del juego la observación que realmente recibe un jugador, es decir, la vista de la configuración real que le llega.
Función heurística	Función que estima qué tan bueno es un estado no terminal, o qué tan cerca está de la victoria, cuando la explosión combinatoria impide explorar el árbol hasta el final.
General Game Player	Agente autónomo que recibe en tiempo de ejecución la descripción formal de un juego que no conocía y lo juega bien, sin ajustes manuales en su código para ese dominio.
Información perfecta	Propiedad de los juegos cuyos conjuntos de información en los nodos de decisión son unitarios: quien decide conoce la historia completa hasta ese nodo. No implica por sí sola determinismo ni ausencia de azar.
Intérprete	Módulo de software que analiza el archivo de reglas externo, valida su estructura y construye en memoria la representación del juego.
Juego bien formado (*well-formed game*)	Juego formal que cumple a la vez con la jugabilidad (no hay bloqueos o *deadlocks*) y con la terminación garantizada en un número finito de jugadas.
Lenguaje de descripción de juegos	Notación formal y declarativa de dominio específico (como GDL o Ludii) con la que se codifican la dinámica, la legalidad y las metas de un juego, separadas del algoritmo de decisión.
Minimax	Algoritmo clásico de decisión para juegos de dos jugadores y suma constante: elige la acción que maximiza la utilidad mínima que puede asegurar frente a un adversario racional.
Motor de decisión	Núcleo del agente que explora las jugadas posibles y elige una acción mediante búsqueda en árboles, podas o simulación estocástica.
Negación estratificada	Propiedad de la semántica de Datalog que prohíbe la recursión cruzada con negación, lo que asegura que el modelo lógico de las reglas sea único, finito y decidible.
Poda alfa-beta	Optimización de Minimax que descarta subárboles completos cuya exploración no puede cambiar la decisión óptima en la raíz.
Recompensa	Valor numérico que recibe cada rol al llegar a una configuración terminal y que mide qué tan bien le fue en la partida.
Relación de indistinguibilidad	Relación de equivalencia sobre los estados que vincula dos configuraciones cuando un participante recibe exactamente la misma observación de ambas.
Relajación de efectos negativos (*delete relaxation*)	Relajación de planificación que elimina efectos de borrado. Para STRIPS proposicional explícito su alcanzabilidad es polinomial, pero el plan relajado óptimo sigue siendo NP-hard; las aproximaciones no son siempre admisibles.

Continúa en la siguiente página...

Término	**Definición**
Retropropagación (*backpropagation*)	Fase del ciclo de MCTS en la que la recompensa obtenida en una hoja o en una simulación se propaga hacia la raíz y actualiza las estadísticas de visitas y el valor medio.
Simulación estocástica (*rollout*)	Fase de MCTS en la que se juega una partida rápida, con movimientos elegidos al azar, desde un nodo expandido hasta una condición terminal.
Transición	Función que, a partir del estado actual y de un vector de acciones conjuntas, determina el único estado sucesor.

Abreviaturas

Abreviatura	Significado
AI	Artificial Intelligence (Inteligencia Artificial)
CWA	Closed World Assumption (Asunción de Mundo Cerrado)
DSL	Domain-Specific Language (Lenguaje de Dominio Específico)
GDL	Game Description Language (Lenguaje de Descripción de Juegos)
GGP	General Game Playing o General Game Player, según el contexto
IA	Inteligencia Artificial
MCTS	Monte Carlo Tree Search (Búsqueda de Árbol Monte Carlo)
RL	Reinforcement Learning (Aprendizaje por Refuerzo)
TT	Trabajo Terminal
UCT	Upper Confidence Bounds applied to Trees

Capítulo 1

Introducción

1.1. Contexto y motivación

Los juegos acompañan a la inteligencia artificial desde sus primeros años. Sirven como banco de pruebas porque tienen reglas explícitas, espacios de estados discretos y resultados que cualquiera puede verificar, algo que el mundo físico rara vez ofrece [1]. En la década de 1950, Arthur L. Samuel escribió un programa para jugar a las damas inglesas que combinaba búsqueda Minimax con una evaluación heurística y que mejoraba con la experiencia acumulada [2]. Con él se popularizó el término *machine learning*, aunque la idea ya la había explorado en teoría Alan Turing [3]; lo que hizo Samuel fue llevarla a la práctica.

Las décadas siguientes dejaron varios hitos frente a jugadores humanos. En 1997, Deep Blue, la supercomputadora de IBM, derrotó al campeón mundial Garry Kasparov [4]. Lo logró con hardware masivamente paralelo y una búsqueda alfa-beta que examinaba hasta doscientos millones de posiciones por segundo, guiada por una función de evaluación pensada y ajustada solo para el ajedrez. Después, el sistema Chinook resolvió el juego de damas [5] y, en años más recientes, AlphaGo alcanzó un desempeño destacado en Go [6]. AlphaZero extendió un mismo algoritmo de aprendizaje y búsqueda a Ajedrez, Shogi y Go, con entrenamiento separado para cada juego [7].

Estos avances tienen distintos alcances de generalidad. Deep Blue se diseñó para Ajedrez y AlphaGo para Go; transferirlos a otro juego requiere adaptar su representación y sus reglas. AlphaZero comparte un algoritmo entre tres dominios, pero utiliza reglas, representaciones de acciones y una instancia entrenada para cada uno [7]. La propuesta de este TT estudiará otra forma de generalidad: recibir reglas externas y construir un modelo consultable sin reprogramar el núcleo de decisión.

Para atacar esta limitación, a partir de 2005 la Universidad de Stanford y la *Association for the Advancement of Artificial Intelligence* (AAAI) impulsaron la línea de investigación conocida como *General Game Playing* (GGP) [8], [9]. Aquí el objetivo ya no es construir el mejor programa para un juego concreto, sino un agente que reciba la descripción formal de un juego que nunca ha visto y lo juegue bien, sin apoyarse en algoritmos o heurísticas escritos para ese dominio [10]. El trabajo de fondo pasa así del programador al agente, que debe interpretar la lógica del juego, armar sus propias representaciones internas y decidir en tiempo real.

1.2. Planteamiento del problema

Los agentes de juego tradicionales dependen demasiado de las reglas del entorno para el que fueron escritos. Sus estructuras de datos se diseñan a la medida del tablero y sus funciones heurísticas incorporan conocimiento empírico del dominio, como el valor relativo de las piezas en el ajedrez o tablas de posiciones ganadoras calculadas de antemano.

En General Game Playing el agente tiene que manejar el espacio de estados $\mathcal{S}$ de cualquier juego que le toque, sin saber de antemano cómo funciona. Desde la teoría de juegos, un juego de información perfecta se modela con una tupla formal: los jugadores, los estados válidos, las acciones permitidas, la función de transición y las utilidades terminales. El problema es el tamaño. Las configuraciones posibles crecen de forma exponencial y a veces ni siquiera se pueden recorrer completas.

Un ejemplo permite distinguir las configuraciones representables de las que satisfacen restricciones. Para las ocho damas se considerarán aquí **configuraciones completas**: un vector $v = (v_1, \ldots, v_8)$ indica la fila de la dama en cada columna de un tablero de 8×8. El espacio de representación es $\mathcal{S}_{\text{rep}} = \{0, \ldots, 7\}^8$, con $8^8 = 16\,777\,216$ elementos. Si se exige además una dama por fila, el subconjunto $\mathcal{S}$ contiene las $8! = 40\,320$ permutaciones. Las soluciones $\mathcal{S}_{\text{solución}} \subset \mathcal{S}$ satisfacen también $|v_i - v_j| \neq |i - j|$ para $i \neq j$, evitando ataques diagonales. Son 92 configuraciones [11]; al agruparlas por rotaciones y reflexiones del cuadrado se obtienen 12 clases. Esta convención clasifica configuraciones completas; no define los estados parciales ni las transiciones de un algoritmo de colocación. En el agente propuesto, las restricciones del dominio deberán proceder de la descripción externa.

En Ajedrez deben distinguirse el espacio de posiciones y el árbol de variantes. Shannon estimó un orden de 10^{43} configuraciones de piezas mediante un conteo combinatorio y un orden de 10^{120} variantes bajo supuestos de ramificación y duración medias [12]. No son un censo exacto de posiciones legales alcanzables ni del árbol completo. Estas estimaciones ilustran por qué el agente deberá generar sucesores a solicitud y decidir bajo límites de búsqueda, en vez de enumerar todo el espacio.

El problema que atiende este Trabajo Terminal es cómo construir una arquitectura desacoplada, basada en reglas externas y en un intérprete, para reconocer acciones legales, aplicar transiciones y seleccionar jugadas bajo un presupuesto de búsqueda. Al cargar otro juego cambiarán la descripción y el modelo derivado de ella, conservando el código del núcleo. La calidad de las decisiones se estudiará con las métricas propuestas, sin anticipar fuerza competitiva ni optimalidad con búsqueda limitada.

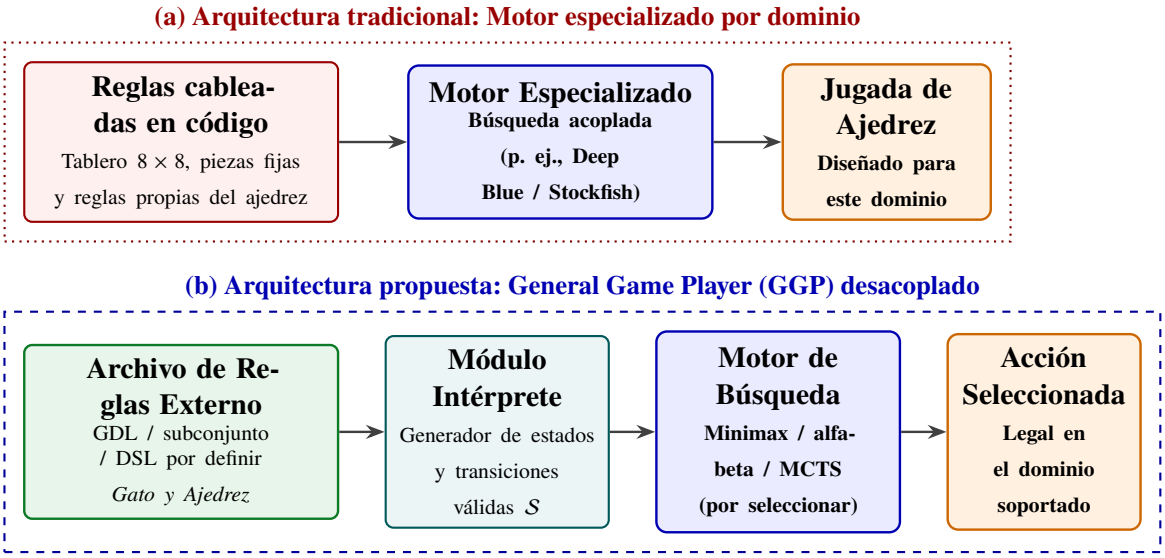


Figura 1.1: Comparación estructural entre un motor especializado tradicional y la arquitectura desacoplada de un General Game Player.

1.3. Justificación

El punto de partida de este trabajo es una incomodidad concreta: casi todos los agentes de juego quedan atados a un solo dominio. Tanto en la industria como en la academia lo habitual es dejar fijos, dentro del código fuente, las reglas, el tablero y las mecánicas. Un General Game Player que no dependa del dominio ofrece otra salida, porque el mismo programa se reutiliza con juegos distintos.

Lo distintivo de la propuesta es que el agente no sabe nada de ningún juego. No juega bien por consejos que dejó un experto humano, sino porque lee la descripción formal de las reglas y aplica procesos generales de búsqueda heurística y simulación. Las transiciones válidas y las condiciones de victoria salen del código ejecutable y pasan a un archivo externo estructurado; con eso se busca que el motor de inteligencia artificial quede separado por completo de la lógica de cada juego.

Desde la Ingeniería en Sistemas Computacionales, el desarrollo reúne varias áreas. Hace falta teoría de lenguajes formales y autómatas para diseñar el análisis léxico, sintáctico y semántico del intérprete de reglas; también modelado matemático y estructuras de datos eficientes para guardar en memoria espacios de estados enormes; y, por último, algoritmos de decisión adversarial, tanto clásicos como modernos, como Minimax con poda alfa-beta y Monte Carlo Tree Search (MCTS).

La viabilidad del proyecto se plantea dentro del ciclo anual de Trabajo Terminal en la Escuela Superior de Cómputo del Instituto Politécnico Nacional, con un alcance académico y búsqueda acotada. Se propone comprobar la reutilización de una arquitectura modular que procese reglas declarativas; igualar los resultados publicados de sistemas de aprendizaje profundo queda fuera de este alcance.

1.4. Objetivos

1.4.1. Objetivo general

Diseñar e implementar un General Game Player (GGP) para juegos de información perfecta, capaz de interpretar las reglas de distintos entornos a partir de un lenguaje de descripción estructurado, basado en un modelo formal de juego. El agente debe ajustar su toma de decisiones a la especificación que se le cargue, sin modificar el código fuente de su motor algorítmico base.

1.4.2. Objetivos específicos

1. Diseñar la sintaxis y la semántica del lenguaje de descripción de reglas, es decir, el formato del archivo.

2. Desarrollar el intérprete y el generador de estados.

3. Implementar el motor de toma de decisiones del agente.

4. Validar la generalización del agente y del formato de archivo con dominios de reglas muy distintos, en particular el Gato (*Tic-Tac-Toe*) y el Ajedrez, de modo que el cambio de comportamiento dependa únicamente del archivo de especificación.

5. Evaluar el desempeño táctico del agente en ajedrez mediante partidas contra un motor especializado con ELO configurable, y obtener métricas como tasa de victorias, longitud de las partidas y tiempo de respuesta por jugada.

1.5. Alcance y limitaciones

Este Trabajo Terminal se limita a juegos que cumplen cuatro condiciones a la vez: información perfecta (el estado completo de la partida y el historial de jugadas son conocidos por todos los participantes en todo momento), estados y acciones discretos, dinámica determinista, es decir, sin azar, y turnos que se alternan de forma secuencial. Con esta frontera el esfuerzo se concentra en el lenguaje de reglas, el intérprete y los algoritmos de búsqueda general, sin sumar la complejidad de los modelos probabilísticos ni de los conjuntos de información parcialmente observables.

Elegimos dos casos que se parecen poco entre sí. El Gato (*Tic-Tac-Toe*) es tan pequeño que puede revisarse entero, y por eso ahí es posible comprobar de forma analítica que el intérprete y las transiciones funcionan bien. El Ajedrez está en el otro extremo: con tantas jugadas posibles y una táctica tan profunda, muestra qué le pasa al motor cuando el problema es grande.

La validación no buscará superar en fuerza de juego a motores especializados como Stockfish, un motor libre y de código abierto para Ajedrez [13]. Tampoco se propone reproducir el entrenamiento profundo de AlphaZero o Student of Games [7], [14]. Los recursos empleados para entrenar esos sistemas y los de su ejecución posterior son aspectos diferentes; aquí se estudiará la generalidad operativa mediante reglas externas y búsqueda acotada, sin atribuir al prototipo un desempeño aún no medido.

1.6. Metodología y planeación del desarrollo

La metodología y el calendario de trabajo se tomarán del protocolo del TT 2027-A178, en particular de sus apartados de productos esperados, metodología y cronograma [15]. Se conservarán las actividades, los responsables y los meses allí establecidos. Para facilitar el seguimiento, se propone complementar esa base con entregables verificables, dependencias y evidencias de aceptación. Estos criterios operativos se distinguirán de los compromisos del protocolo y no se interpretarán como resultados ya obtenidos.

1.6.1. Enfoque iterativo e incremental

El desarrollo se organizará mediante ciclos de trabajo en los que se definirá un objetivo acotado, se elaborará un entregable y se revisará su correspondencia con las reglas y el modelo formal del juego. Las observaciones de cada revisión se utilizarán para ajustar el siguiente ciclo. De esta manera, la integración del sistema se realizará de forma gradual y permitirá identificar inconsistencias entre la descripción de reglas, la interpretación del estado y la selección de acciones.

En cada ciclo se propone registrar la versión del entregable, las comprobaciones realizadas y las limitaciones pendientes. Una actividad se considerará concluida únicamente cuando exista evidencia revisable de su cumplimiento. La presencia de una actividad en el cronograma, por sí sola, no acreditará su ejecución ni un porcentaje de avance.

1.6.2. Etapas de desarrollo

Se recuperarán las cuatro etapas metodológicas del protocolo [15]:

1. **Diseño del lenguaje de descripción de reglas.** Se definirán la sintaxis, la gramática y la semántica necesarias para expresar estados, acciones, transiciones y condiciones terminales. La elección entre GDL completo, un subconjunto o un DSL propio deberá documentarse antes de cerrar la especificación.

2. **Construcción del módulo intérprete.** Se desarrollará y validará el procesamiento del archivo de reglas y la representación interna del juego. Se verificará que las consultas de legalidad, transición y terminalidad correspondan a la especificación definida.

3. **Implementación del agente.** Se desarrollará la toma de decisiones sobre el modelo que exponga el intérprete. La selección de algoritmos, las funciones heurísticas y los presupuestos de búsqueda se documentarán sin introducir reglas particulares de Gato o Ajedrez dentro del núcleo de decisión.

4. **Integración y validación experimental.** Se integrarán el intérprete, el modelo del juego y el agente. Se prepararán los archivos de los dos dominios y se ejecutarán los experimentos que se definan para comprobar legalidad, funcionamiento y desempeño bajo condiciones reproducibles.

Estas etapas expresarán dependencias de trabajo y podrán revisitarse durante las iteraciones. No implicarán que se deba enumerar en memoria la totalidad del espacio de estados de Ajedrez ni que el agente seleccione una jugada óptima cuando la búsqueda sea limitada.

1.6.3. Productos esperados

Se conservarán los cinco productos establecidos en el protocolo [15]:

1. **Código fuente del General Game Player:** incluirá el intérprete del archivo de reglas y el motor algorítmico del agente.
2. **Especificación formal del lenguaje de reglas:** documentará la sintaxis, la semántica y la gramática del formato adoptado.
3. **Archivos de entorno de Gato y Ajedrez:** describirán los dos dominios que se utilizarán en la validación, con las restricciones que se acuerden para cada uno.
4. **Reporte de pruebas y experimentos:** presentará los escenarios, las condiciones de ejecución, las métricas y los resultados efectivamente obtenidos.
5. **Manual de usuario:** explicará cómo ejecutar el agente y cómo escribir descripciones compatibles con el lenguaje especificado.

1.6.4. Matriz de seguimiento y evidencias propuestas

La Tabla 1.1 relaciona las etapas con sus entregables y dependencias. Las evidencias de aceptación son una propuesta operativa para el seguimiento; no se atribuyen literalmente al protocolo ni constituyen pruebas ya realizadas. Los umbrales cuantitativos se definirán al diseñar los experimentos.

Tabla 1.1: Matriz propuesta de seguimiento del desarrollo.

Etapa	Actividad	Entregable	Dependencia	Evidencia de aceptación propuesta
1. Lenguaje	Definir gramática y semántica de las reglas.	Especificación y ejemplos de descripciones.	Modelo formal y alcance de juegos definidos.	Se explicará cómo se representan estados, acciones, legalidad, transiciones, terminales y utilidades, sin reglas implícitas en el agente.
2. Intérprete	Procesar reglas y construir el modelo consultable.	Intérprete y casos de comprobación.	Especificación de la etapa 1.	Se contrastarán las acciones legales, los estados sucesores y la terminalidad con casos cuyo resultado esperado se haya documentado previamente.

Continúa en la siguiente página.

Tabla 1.1 (continuación).

Etapa	Actividad	Entregable	Dependencia	Evidencia de aceptación propuesta
3. Agente	Seleccionar acciones mediante búsqueda general.	Motor de decisión y registro de ejecución.	Consultas del intérprete verificadas.	Se comprobará que las acciones elegidas sean legales y que la decisión respete el presupuesto de búsqueda definido, registrando tiempo y nodos explorados.
4. Validación	Integrar módulos y ejecutar experimentos.	Archivos de juegos, reporte y manual.	Integración funcional y diseño experimental definido.	Se documentará qué cambia al cargar otro juego; se verificará que el núcleo se conserve y se reportarán resultados, condiciones de ejecución y limitaciones.

1.6.5. Diseño de la evaluación: métricas y muestra

Se propone evaluar la corrección de las reglas, la calidad de decisión, el costo de búsqueda y la generalidad operativa de la arquitectura de la Sección 5.1. Se documentarán los casos, las referencias de comparación y el procedimiento antes de realizar las mediciones. Este diseño complementará los objetivos existentes y mantendrá pendiente el acuerdo sobre el experimento definitivo de Ajedrez. Las posiciones funcionales y los problemas de primera jugada serán comprobaciones complementarias: no sustituirán las partidas contra un motor con ELO configurable previstas en el objetivo específico. Su motor, nivel, número de partidas y presupuesto continuarán pendientes de acuerdo.

Métricas y procedimiento de medición

Para cada comprobación se preparará una entrada y un resultado esperado obtenido de una referencia independiente. Para los indicadores de acierto de movimientos, transiciones, terminalidad, utilidad y decisión se utilizará la proporción de la Ecuación (1.1):

$$P_k = 100\,\frac{C_k}{N_k}, \qquad N_k > 0, \tag{1.1}$$

donde N_k será el número de casos evaluados del indicador k y C_k el número de coincidencias. Los denominadores se reportarán por separado: posiciones, pares estado–acción, estados terminales o decisiones, según corresponda. El tiempo, la profundidad y los nodos se expresarán en sus unidades propias; la generalidad se comprobará mediante evidencia de conservación del núcleo. La Tabla 1.2 precisa cada comprobación.

Tabla 1.2: Métricas y procedimientos propuestos de evaluación.

Aspecto	Procedimiento e indicador
Movimientos legales	Porcentaje de posiciones cuyo conjunto completo de acciones coincida con la referencia. Se registrarán por separado las acciones ilegales generadas y las legales omitidas; igualar únicamente la cantidad no será suficiente.
Transiciones	Porcentaje de pares estado–acción cuyo sucesor coincida con el esperado. Se compararán la configuración, el turno y los datos adicionales exigidos por las reglas, incluidos los antecedentes pertinentes.
Terminalidad y utilidad	Dos porcentajes independientes: clasificación correcta de terminalidad sobre todos los estados evaluados y asignación correcta del vector de utilidades sobre los estados terminales.
Calidad de decisión	En Gato, porcentaje de decisiones pertenecientes al conjunto óptimo calculado por un solucionador independiente con Minimax. En Ajedrez, porcentaje de aciertos de primera jugada: la acción inicial deberá pertenecer al conjunto de respuestas aceptadas y entregarse dentro del presupuesto acordado. Las acciones inválidas y los tiempos agotados contarán como fallos.
Costo de búsqueda	Tiempo por decisión en milisegundos, resumido mediante mediana y percentil 95; número de nodos expandidos y profundidad máxima del árbol explorado, contada en medias jugadas desde la raíz (profundidad cero). Se contará la profundidad de los nodos incorporados al árbol; el horizonte completado de Minimax y la longitud de las simulaciones de MCTS se reportarán aparte cuando correspondan. Se medirá desde la solicitud de acción sobre un estado disponible hasta la respuesta; la carga del archivo se registrará aparte.
Generalidad operativa	Se ejecutarán ambos dominios con la misma versión del intérprete y del motor. Se registrarán los componentes modificados y se revisará que las reglas particulares permanezcan en las descripciones externas. El criterio será conservar el núcleo al cambiar de juego y verificar su funcionamiento.

Para Ajedrez se podrá utilizar una implementación independiente, como *python-chess*, para contrastar movimientos, sucesores y condiciones terminales [16]. Su uso como referencia de pruebas no sustituirá las consultas del intérprete propuesto. Se fijarán su versión y las reglas comparadas. Se propone exigir coincidencia del 100 % en los casos funcionales incluidos; esta condición no demostrará corrección fuera del banco. Los umbrales de calidad y tiempo se justificarán después del piloto.

Selección de la muestra y de los casos

En Gato se propone una revisión exhaustiva de los estados alcanzables desde el tablero vacío, con X al inicio y detención al aparecer una victoria o un empate. Se incluirán los estados terminales para comprobar sus resultados y los no terminales para evaluar decisiones. Se conservarán las configuraciones simétricas como casos distintos y se verificará la enumeración mediante una referencia independiente.

Para Ajedrez se propone un banco piloto de **60 posiciones funcionales**, organizado inicialmente en seis familias de diez casos: movimientos y capturas, jaques y clavadas, enroques, promociones, captura al paso y terminalidad. Cada caso tendrá una familia principal; se cubrirán ambos colores y situaciones que admitan o impidan la acción. La composición y el tamaño se ajustarán cuando se acuerden las reglas soportadas.

Se propone además reservar **30 problemas tácticos para evaluar la primera jugada**, seleccionados por tema y dificultad, con respuestas verificadas. Una fuente posible será la base abierta de Lichess [17]; se preparará la posición aplicando a su FEN el primer movimiento proporcionado, pues la solución comienza con el segundo. Se documentará por caso el conjunto de primeras jugadas aceptadas, incluidas las alternativas válidas. El piloto y los ajustes de parámetros utilizarán casos distintos de estos 30. El indicador medirá el acierto inicial, sin atribuir la resolución de la secuencia completa.

Las cantidades de 60 y 30 son propuestas exploratorias de cobertura, no tamaños que garanticen representatividad estadística de todo Ajedrez. Cada caso conservará identificador, procedencia, posición, regla evaluada y resultado esperado. Las repeticiones de una misma posición se contabilizarán como ejecuciones y no como nuevas posiciones.

Condiciones de comparación y registro

Las configuraciones se compararán sobre los mismos casos y bajo un presupuesto de búsqueda acordado. Se documentarán el algoritmo, los parámetros, la versión de las reglas, los criterios de parada y el tratamiento de los estados terminales. Se contará una expansión cuando se generen sucesores de un nodo; las expansiones y las simulaciones tendrán contadores separados. Si se utiliza aleatoriedad, se repetirán las ejecuciones con semillas registradas; el número definitivo de repeticiones se justificará con la variabilidad observada en el piloto.

Los registros incluirán caso, configuración, repetición, acción elegida, resultado de la comprobación, tiempo, nodos y motivo de terminación. En calidad de decisión, cada combinación de posición y ejecución será un ensayo; se informarán tanto el número de posiciones distintas como el de ensayos por configuración. Los fallos y agotamientos de tiempo permanecerán en el informe, sin excluirlos del denominador de decisiones. La evidencia de generalidad se limitará a los dominios y reglas efectivamente evaluados.

1.6.6. Distribución del trabajo y calendario

Se mantendrá la distribución del protocolo: Sánchez Catarino Edgar se ocupará de la fundamentación del motor, la representación matemática, las reglas y la generación de movimientos, la implementación e integración del agente y el archivo de descripción de Ajedrez. Linares Villegas Gustavo se ocupará del estado del arte y la definición del problema, el diseño de árboles de búsqueda y la función heurística, los experimentos, el análisis estadístico y las conclusiones. Ambos participarán en la redacción de capítulos técnicos y manuales, la revisión final y la preparación de la defensa [15].

Los cronogramas individuales se presentan en el apartado de Cronogramas, dentro de Anexos. Se conservará la secuencia de agosto a junio y el sombreado de las actividades del documento base, sin agregar años, fechas de entrega ni porcentajes de avance que no estén confirmados.

Se deberá revisar una dependencia del calendario: los experimentos de Gustavo están previstos para febrero y marzo, mientras que el archivo de Ajedrez de Edgar está previsto para abril. Antes de ejecutar la validación, se determinará qué experimentos podrán realizarse con los recursos disponibles y cuáles requerirán ese archivo. Esta observación no modifica los periodos del protocolo ni supone que ya exista una descripción parcial de Ajedrez.

Capítulo 2

Estado del arte

2.1. Evolución de los sistemas de juego en inteligencia artificial

En ciencias de la computación, el estudio de los juegos ha seguido dos caminos. El primero, y el más antiguo, es construir motores muy especializados que rindan al máximo en un solo dominio. El segundo, que llegó después, busca agentes generales que entiendan y jueguen distintos juegos a partir de descripciones abstractas [1].

Al principio de esta historia, en 1959, Arthur Samuel mostró que una computadora podía mejorar en las damas inglesas mediante aprendizaje inductivo y una evaluación heurística de posiciones apoyada en Minimax [2]. Su trabajo abrió una larga tradición en la que cada juego se atacaba por separado, afinando a mano las funciones de evaluación y explorando árboles de juego de forma masiva.

Ese enfoque alcanzó su cima en 1997, cuando Deep Blue venció al campeón mundial Garry Kasparov [4]. Combinaba chips hechos a la medida para generar jugadas de ajedrez con una búsqueda alfa-beta muy paralelizada, que revisaba hasta doscientos millones de posiciones por segundo. Su criterio, sin embargo, lo habían calibrado a mano Grandes Maestros: pesos amarrados a las sesenta y cuatro casillas del tablero y al valor de cada pieza. Con Chinook pasó algo parecido en las damas, que resolvió en 2007 apoyándose en bases de datos exhaustivas de finales [5]. Y TD-Gammon llegó a nivel de maestro en backgammon con redes neuronales entrenadas por diferencias temporales.

Desde 2016 llegó otro salto, con AlphaGo [6], AlphaZero [7] y MuZero [18]. AlphaZero mostró que bastaban las reglas básicas para dominar el ajedrez, el Shogi y el Go. Lo hizo uniendo la búsqueda en árboles Monte Carlo (MCTS) con redes neuronales profundas entrenadas por autojuego, sin aperturas ni conocimiento humano experto.

En esta revisión se distinguirá la reutilización de un algoritmo de aprendizaje de la interpretación de reglas externas. AlphaZero comparte aprendizaje y búsqueda entre juegos, pero emplea reglas y codificaciones de estados y acciones propias de cada dominio, además de entrenar una instancia por juego [7]. GGP plantea recibir una descripción formal y razonar sobre ella [8]. Esta comparación precisa el sentido de generalidad que interesa al TT, sin negar la aportación de AlphaZero ni atribuirle una interfaz declarativa que su publicación no propone.

2.2. Surgimiento y fundamentos de General Game Playing

Para atacar de raíz la hiperespecialización, en 2005 la Universidad de Stanford y la Asociación para el Avance de la Inteligencia Artificial (AAAI) lanzaron la iniciativa de *General Game Playing* (GGP) y convocaron a la primera competencia internacional abierta [8]. El documento fundacional, de Genesereth, Love y Pell, planteaba construir agentes autónomos que reciban la especificación formal de un juego cualquiera, desconocido de antemano, y lo jueguen bien sin supervisión humana ni cambios en su código.

Lo que hizo posible este esquema fue el *Game Description Language* (GDL) [9], [10]. El lenguaje se inspira en la programación lógica declarativa, en concreto en el subconjunto de Datalog con negación estratificada, y establece un compromiso claro: todo juego discreto, finito, síncrono y de información perfecta puede describirse como una máquina de estados abstracta usando unos pocos predicados lógicos.

Así transcurre una partida típica de GGP. El agente se conecta a un servidor central, que le manda el archivo de reglas unos minutos antes de empezar. Ese margen lo marca un reloj de preparación (*start clock*) y sirve para procesar la gramática, analizar el juego y armar las representaciones internas. Una vez que arranca la partida, cada turno tiene un reloj de jugada (*play clock*) para explorar el espacio de estados y responder. Con estas condiciones no hay lugar para heurísticas escritas de antemano: el agente tiene que derivar qué movimientos son legales, prever qué provocan y estimar qué tan cerca está del final, todo con la semántica de las reglas que recibió.

2.3. Generaciones y agentes emblemáticos de General Game Playing

Para organizar esta revisión se agrupan los agentes en tres generaciones expositivas: análisis de reglas, búsqueda por simulación y optimización de la simulación. Esta división servirá para comparar enfoques; no se atribuye a la literatura una taxonomía única ni fronteras cronológicas estrictas. Los trabajos originales sustentan las técnicas descritas [19], [20], [21], [22].

2.3.1. Primera generación: Análisis estático de reglas y Minimax heurístico

En los primeros años, la idea fue llevar al caso general las técnicas clásicas de búsqueda adversarial, Minimax y poda alfa-beta. El problema es que, sin conocer el juego, el agente no tenía una función heurística con la cual evaluar los estados donde el árbol se corta por profundidad.

ClunePlayer, de Jim Clune en la Universidad de California en Los Ángeles, fue el primer campeón de GGP, en la competencia de la AAAI de 2005 [19]. Su aporte fue analizar la estructura sintáctica de las reglas GDL durante el tiempo de preparación para construir automáticamente una función de evaluación posicional. Identificaba heurísticas generales: la estabilidad de hechos (fluents que, una vez alcanzados, ya no pueden destruirse), la movilidad (cuántas jugadas legales tiene cada jugador) y la distancia simbólica a las condiciones de victoria definidas en las metas.

En 2006 el campeón fue Fluxplayer, de Stephan Schiffel y Michael Thielscher, en la Universidad Técnica de Dresde [20]. En lugar de inferencia lógica directa, usaba un razonador basado en el cálculo de fluents (*Fluent Calculus*) junto con aproximaciones de lógica difusa. Así podía

medir cuán cerca estaba un estado cualquiera de cumplir las condiciones de victoria, y esas evaluaciones continuas alimentaban una búsqueda alfa-beta con profundización iterativa. Ambos sistemas tenían, sin embargo, un mismo punto débil: cuando las reglas eran muy combinatorias o no lineales, analizar las cláusulas resultaba muy costoso y producía heurísticas engañosas que empeoraban las decisiones.

2.3.2. Segunda generación: La revolución de Monte Carlo Tree Search

El paso a la segunda generación cambió el campo por completo. Como era difícil inferir heurísticas estáticas confiables, los investigadores empezaron a probar métodos basados en simulación estocástica y en árboles de decisión que se adaptan sobre la marcha.

En 2007, Hilmar Finnsson y Yngvi Björnsson, de la Universidad de Reikiavik, presentaron CadiaPlayer [21]. CadiaPlayer dejó de lado el Minimax guiado por heurísticas sintácticas y aplicó por primera vez en GGP el algoritmo *Monte Carlo Tree Search* (MCTS) con la política de balance UCT (*Upper Confidence Bounds applied to Trees*) [23]. Lo interesante es que el agente no necesitaba entender a fondo las reglas para valorar una posición: simulaba cientos de partidas aleatorias desde el nodo actual hasta el final y promediaba las recompensas para decidir hacia dónde explorar. Ganó el campeonato mundial en 2007, 2008 y de nuevo en 2012, y desde entonces MCTS domina en General Game Playing.

En 2009 el primer lugar fue para Ary, de Jean Méhat en la Universidad de París 8, que implementó un motor MCTS muy paralelizado: corría simulaciones en hilos concurrentes y podaba de forma dinámica las ramas que resultaban estadísticamente desfavorables durante el muestreo.

En esa época apareció también una innovación clave en la representación interna de los estados: las redes proposicionales o PropNets. En vez de resolver consultas lógicas en cada nodo del árbol con un motor de Datalog, las reglas se compilaban en la fase de preparación como una red de compuertas booleanas (`AND`, `OR`, `NOT`) alojada directamente en memoria. Calcular el siguiente estado dejaba de ser un problema de resolución lógica y pasaba a ser la propagación de bits por un circuito, con lo que un motor MCTS podía correr varios órdenes de magnitud más simulaciones por segundo. La Figura 2.1 contrasta la deducción lógica interpretada con la aceleración por propagación de compuertas booleanas.

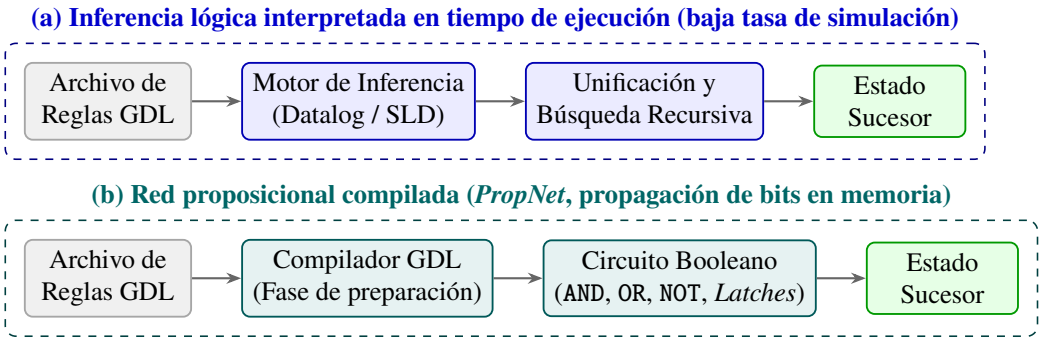


Figura 2.1: Comparación esquemática entre la inferencia lógica interpretada recursivamente y la evaluación acelerada por propagación de señales en redes proposicionales (*PropNets*).

2.3.3. Tercera generación: PropNets optimizadas y heurísticas de simulación

La tercera generación se dedicó a pulir el motor de simulación. El objetivo era superar la lentitud de las partidas puramente aleatorias en juegos con espacios de búsqueda inmensos o ramas terminales muy profundas.

Sancho, desarrollado por Steve Draper y Andrew Rose, ganó la competencia internacional de GGP de 2014, según la página de sus autores [24]. Este antecedente se distinguirá de las técnicas que se describen en publicaciones independientes: MAST utiliza estadísticas históricas de acciones para orientar las simulaciones [21], mientras RAVE reutiliza estimaciones de acciones observadas durante la búsqueda [25]. Su estudio no implica que ambas técnicas estuvieran presentes en Sancho ni que se implementarán en este TT.

La Figura 2.2 resume cómo se pasa de una generación a otra y cómo todas desembocan en los formalismos actuales.

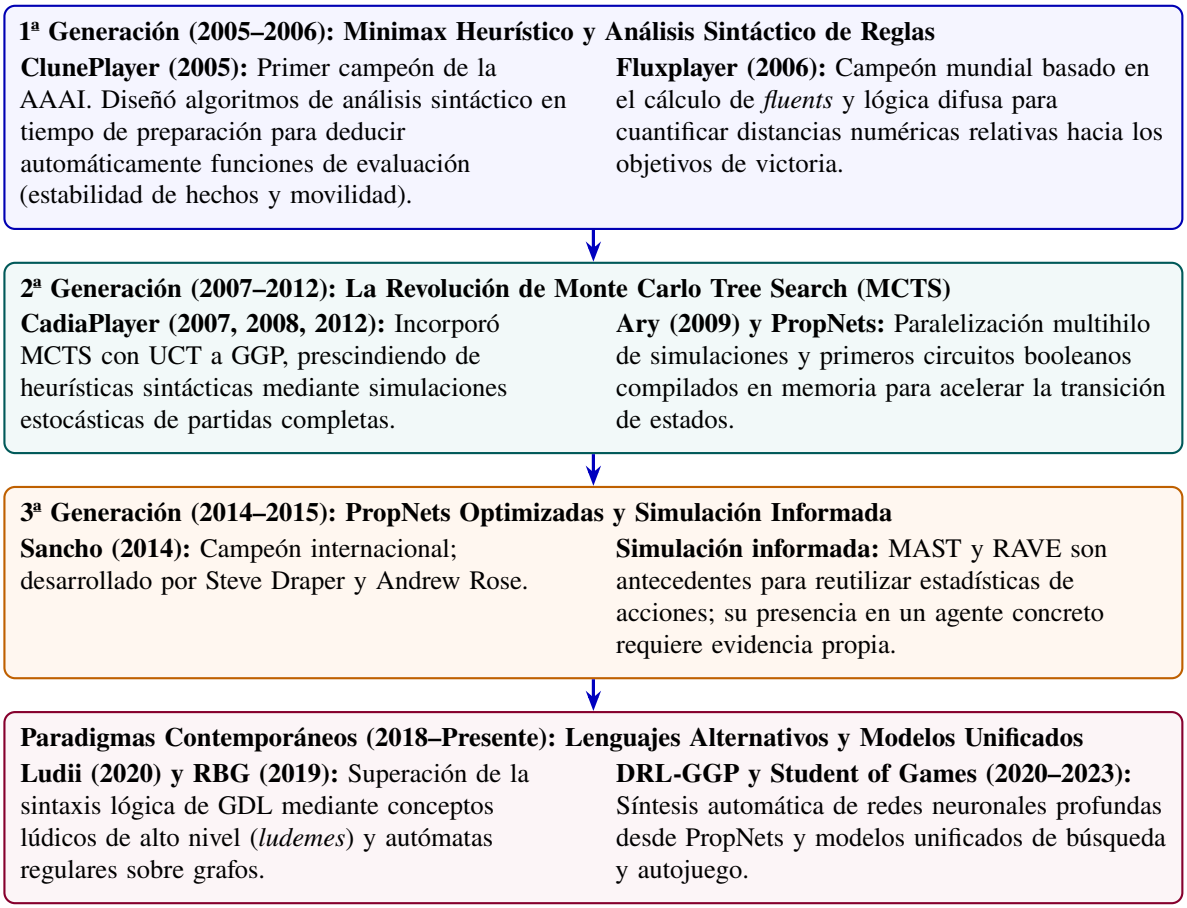


Figura 2.2: Organización expositiva de antecedentes y generaciones algorítmicas en *General Game Playing* (GGP).

2.4. Plataformas y lenguajes modernos de descripción de juegos

GDL clásico llegó a ser el estándar académico para juegos discretos de información perfecta. Con su sintaxis prefija, heredada de KIF, y su semántica puramente declarativa, sin embargo, se volvía lento en tableros grandes o con mecánicas complejas. En la última década han aparecido plataformas y lenguajes alternativos que responden a esa limitación y han ampliado lo que se investiga.

2.4.1. Extensiones de GDL e información imperfecta

Para cubrir juegos con azar y con información oculta, Michael Thielscher formalizó en 2010 la extensión GDL-II (*Game Description Language for Incomplete Information and Stochastic Games*) [26]. Añadió al lenguaje los eventos estocásticos, mediante un rol especial llamado `random` que emite jugadas con distribución uniforme, como al tirar un dado, y la visibilidad parcial, mediante el predicado `sees`, que modela la función de observación $O_i : S \to \Omega_i$ para ocultar cartas o piezas al adversario. La expresividad creció bastante, pero también la complejidad del razonador, pues los agentes deben rastrear conjuntos de información de Kuhn y sostener creencias probabilísticas sobre los estados que no ven.

2.4.2. El sistema Ludii

Ludii se desarrolló en la Universidad de Maastricht bajo la dirección de Cameron Browne y es uno de los avances más notables en la descripción general de juegos de mesa modernos [27], [28]. En lugar de la lógica de predicados pura de GDL, usa *ludemes*, unidades conceptuales que agrupan información lúdica de alto nivel.

El lenguaje incorpora conceptos de tableros, piezas, movimientos y condiciones de fin. Piette y colaboradores compararon Ludii con un razonador GDL basado en PropNet en los juegos de Tiltyard: reportaron mayor velocidad de simulación para las descripciones y condiciones evaluadas, sin demostrar una superioridad para todo juego o implementación [27]. Soemers y colaboradores documentan más de mil descripciones en la biblioteca de su estudio de 2024 [28]; esa cifra corresponde a la publicación, no a un inventario actualizado del sistema.

2.4.3. Regular Boardgames

Regular Boardgames (RBG) es una propuesta de Jakub Kowalski, Maksymilian Mika, Jakub Sutowicz y Marek Szykuła, presentada en la conferencia AAAI de 2019. Se trata de un formalismo para juegos de tablero que se construye sobre lenguajes regulares y autómatas finitos [29].

En RBG, las reglas de movimiento y las condiciones de legalidad de cada pieza se escriben como expresiones regulares que describen trayectorias válidas sobre una cuadrícula vista como un grafo conexo. Los autores probaron que RBG es universal para juegos finitos, deterministas, por turnos y de información perfecta. Lo más llamativo es su rendimiento: los analizadores basados en autómatas generan movimientos legales y aplican transiciones entre dos y tres órdenes de

magnitud más rápido que los razonadores lógicos de GDL, lo que lo hace muy adecuado para experimentos intensivos de búsqueda y de aprendizaje por refuerzo.

2.4.4. Entornos generales para aprendizaje: OpenSpiel

En el campo del aprendizaje automático actual, DeepMind liberó en 2019 la plataforma de código abierto OpenSpiel [30]. No es un lenguaje declarativo de reglas pensado para usuarios finales, sino un marco algorítmico en C++ y Python con una colección común de juegos e interfaces estándar para investigar aprendizaje por refuerzo, teoría de juegos algorítmica y búsqueda en árboles. Su valor está en ofrecer implementaciones de referencia de algoritmos modernos, como *Counterfactual Regret Minimization*, *Policy Gradients* y MCTS, con lo que se pueden comparar agentes en condiciones metodológicas homogéneas.

2.5. Tendencias contemporáneas en aprendizaje profundo y modelos unificados

Últimamente la investigación intenta juntar dos cosas: la descripción declarativa de juegos y las técnicas de aprendizaje profundo que transformaron a los motores especializados.

Goldwaser y Thielscher estudiaron el aprendizaje por refuerzo profundo en GGP [31]. Su modelo recibe valores de nodos de una PropNet y construye automáticamente capas completamente conectadas, con salidas de política y valor por rol. El algoritmo 1 de su artículo determina tamaños de capas a partir de las entradas y acciones; no reproduce la topología de compuertas como una red convolucional. Los autores explican que las convoluciones requieren un orden geométrico que GDL no proporciona necesariamente. El entrenamiento utiliza autojuego y MCTS; se trata de un antecedente y no de una arquitectura neuronal seleccionada para este TT.

Student of Games, publicado por Schmid y colaboradores en 2023, combina búsqueda, aprendizaje por autojuego y razonamiento de teoría de juegos para estudiar Ajedrez, Go, póker y Scotland Yard [14]. La publicación distingue configuraciones de entrenamiento y de evaluación. Sus recursos y tiempos experimentales no se presentarán como un requisito universal para ejecutar cualquier agente basado en ese enfoque. En este TT sirve como antecedente de generalidad algorítmica; su aprendizaje profundo no forma parte de la implementación comprometida.

2.6. Síntesis comparativa y posicionamiento del Trabajo Terminal

Después de este recorrido se pueden comparar los enfoques principales de la literatura, como se resume en la Tabla 2.1.

Tabla 2.1: Comparación de paradigmas y sistemas representativos en juegos e inteligencia artificial.

Sistema / Paradigma	**Entrada de reglas**	**Motor de decisión**	**Tipo de juego**	**Dependencia de infraestructura**
Deep Blue [4]	Código nativo cableado	Alfa-beta especializado	Ajedrez (info. perfecta)	Hardware dedicado a la medida
AlphaZero [7]	Reglas y codificación por dominio	MCTS + Redes profundas	Varios (info. perfecta)	Entrenamiento y evaluación con recursos distintos
ClunePlayer [19]	Reglas lógicas (GDL)	Minimax con heurística aprendida	General GGP (info. perfecta)	Estación de trabajo estándar
CadiaPlayer [21]	Reglas lógicas (GDL)	MCTS con UCT sin heurística	General GGP (info. perfecta)	Estación de trabajo estándar
Sancho [24]	Reglas lógicas (GDL)	Agente de competencia GGP	General GGP (info. perfecta)	Estación de trabajo estándar
Ludii [27], [28]	Sintaxis de ludemes	MCTS / UCT / Alpha-beta	General (tableros y cartas)	Entorno Java de escritorio
RBG [29]	Expresiones regulares	Búsqueda con autómatas finitos	General (tableros)	Entorno C++ de alto rendimiento
Student of Games [14]	Reglas y codificación por dominio	Búsqueda + Teoría de juegos	Perfecta e imperfecta	Configuraciones experimentales por juego
Presente TT (Propuesta)	**Lenguaje declarativo (DSL/GDL)**	**Minimax / Alfa-Beta / MCTS desacoplado**	**General (información perfecta)**	**Estación de trabajo estándar**

Del análisis se desprende dónde se ubica este Trabajo Terminal.

Primero, se propone aplicar el principio de General Game Playing: el motor de decisión deberá operar sobre el modelo construido a partir de reglas externas, sin variables ni heurísticas codificadas para un dominio particular.

Segundo, el TT se centrará en la interpretación declarativa y la búsqueda acotada. La comparación con AlphaZero y Student of Games no implica que sus costos de entrenamiento sean los mismos que los de tomar decisiones con un modelo entrenado. La generalidad operativa del sistema propuesto se comprobará conservando el núcleo y cambiando la descripción externa.

Por último, se propone una representación interna consultable y una separación entre reglas y decisión. El lenguaje y los algoritmos definitivos requerirán justificar sus alternativas; la presencia de GDL, Minimax o MCTS en esta revisión no los convierte en decisiones de implementación. Con ello se busca fundamentar un agente general dentro del alcance documentado.

Capítulo 3

Marco Teórico

3.1. Inteligencia artificial en juegos

Los juegos se han usado una y otra vez como banco de pruebas para la inteligencia artificial, porque tienen reglas explícitas, objetivos medibles y resultados que se pueden verificar. Desde los primeros trabajos de Arthur Samuel sobre aprendizaje automático en el juego de damas [2], han servido para estudiar búsqueda, planeación, toma de decisiones, aprendizaje y evaluación de estrategias.

Hay otra razón por la que los juegos importan: en un entorno controlado reúnen problemas que aparecen también fuera de ellos. Hay que elegir acciones bajo restricciones, anticipar consecuencias, competir con otros agentes, repartir el cómputo y decidir sin haber explorado todas las posibilidades. En un juego pequeño como el Gato todavía se puede recorrer buena parte del espacio de estados; en el Ajedrez, el número de estados y de secuencias de jugadas crece de forma combinatoria y sin búsqueda ni evaluación no hay manera de avanzar.

Históricamente, muchos de los sistemas más exitosos han sido motores especializados. Rinden bien porque llevan dentro conocimiento del dominio: reglas programadas para un juego, funciones de evaluación ajustadas a sus piezas o posiciones, representaciones hechas a la medida y optimizaciones para un tablero concreto. Eso da agentes fuertes, pero rígidos. Un programa hecho para jugar Ajedrez no puede, sin más, jugar Damas, Go o Gato, y no por falta de capacidad de cálculo, sino porque lo que sabe está pegado al juego para el que se construyó.

De ahí nace el interés por agentes que jueguen más de un juego sin reprogramar el motor, y esa línea de investigación es *General Game Playing*. Una parte del trabajo intelectual se mueve del programador al agente: ya no se espera que alguien codifique cada regla a mano, sino que el agente sepa leer la descripción formal del juego y razonar sobre ella.

3.2. Juegos de información perfecta

En teoría de juegos y en inteligencia artificial [1], [32], [33], algo que separa a los juegos es cuánto ven los jugadores del entorno. En un juego de información perfecta, quien debe decidir conoce la historia completa del juego hasta ese nodo de decisión. Sus conjuntos de información son unitarios. Esta propiedad no exige por sí sola estados discretos, ausencia de azar ni una

determinada alternancia; esas restricciones se adoptan adicionalmente en el alcance de este proyecto. El Ajedrez, las Damas y el Gato son los ejemplos de siempre.

Para formalizar esta idea con el modelo de juego (Γ) de la Sección 3.4, consideremos un conjunto de jugadores $R = \{r_1, r_2, \ldots, r_n\}$ y un conjunto de estados posibles S, donde cada estado $s \in S$ es la configuración completa y real del juego en un instante dado.

3.2.1. Espacio de observaciones y función de observación (O_i)

Conviene distinguir entre lo que el entorno es realmente y lo que cada jugador alcanza a percibir de él. Llamamos **estado real** ($s \in S$) a la configuración completa y verdadera del juego, que incluye todo lo que existe en ese instante: la posición de cada pieza, las cartas que cada participante tiene en la mano (también las que los demás no ven), el tiempo que queda en el reloj y a quién le toca mover. La **observación** ($o \in \Omega_i$), por su parte, es la información, parcial o total, que el jugador r_i recibe de ese estado, es decir, lo que ve en su pantalla o sobre la mesa. El conjunto Ω_i reúne todas las observaciones posibles para ese jugador.

La relación entre la realidad del juego y lo que ve el participante se modela con la **función de observación** [1], [26]:

$$O_i : S \to \Omega_i \tag{3.1}$$

que toma el estado real $s \in S$ y devuelve la observación $O_i(s) \in \Omega_i$ que ve el jugador r_i. Se puede pensar en O_i como un filtro colocado desde el punto de vista del jugador: aunque el juego esté en la configuración completa s, solo pasa lo que las reglas le permiten ver. En un juego de cartas, por ejemplo, el filtro borra las cartas del adversario y deja las propias y las comunitarias.

3.2.2. Relación de indistinguibilidad ($\sim_i$)

A partir de la función de observación se puede establecer con precisión cuándo dos situaciones distintas del juego resultan indiscernibles para un participante [33].

Definición 3.1: Relación de indistinguibilidad

Dado un conjunto de estados S, un espacio de observaciones Ω_i y una función de observación $O_i : S \to \Omega_i$ [26], [32], [33], la **relación de indistinguibilidad** $\sim_i \subseteq S \times S$ para el jugador $r_i \in R$ se define como:

$$s \sim_i s' \iff O_i(s) = O_i(s') \tag{3.2}$$

Dos estados $s, s' \in S$ son indistinguibles para el jugador r_i si y solo si producen exactamente la misma proyección perceptual en su codominio de observaciones.

Que dos estados s y s' sean indistinguibles para un jugador quiere decir que lo que los diferencia queda fuera de su campo de visión. Piénsese en un juego de cartas: en el estado s el adversario tiene un As oculto y en s' tiene un Rey oculto. El adversario los distingue sin problema, porque ve su propia mano. Para el jugador r_i, en cambio, la carta del rival está boca abajo y recibe exactamente la misma vista en ambos casos, $O_i(s) = O_i(s')$, de modo que no puede saber en cuál de las dos situaciones se encuentra.

3.2.3. La indistinguibilidad como relación de equivalencia

Para agrupar los estados que un jugador confunde entre sí hay que demostrar que $\sim_i$ es una relación de equivalencia sobre el espacio de estados [33].

Proposición 3.1. *Para todo jugador $r_i \in R$, la relación binaria de indistinguibilidad $\sim_i$ es una relación de equivalencia sobre el conjunto de estados S.*

Demostración (véase [33]). Se debe verificar que $\sim_i$ satisface las tres propiedades fundamentales:

1. **Reflexividad ($\forall s \in S : s \sim_i s$):** Sea $s \in S$ un estado arbitrario. La observación que produce dicho estado es idéntica a sí misma:

   $$O_i(s) = O_i(s)$$

   Por tanto, $s \sim_i s$ para todo $s \in S$.

   Un estado es indistinguible de sí mismo bajo la misma función de observación.

2. **Simetría ($\forall s, s' \in S : s \sim_i s' \implies s' \sim_i s$):** Sean $s, s' \in S$ tales que $s \sim_i s'$. Por definición, ambos estados producen la misma observación: $O_i(s) = O_i(s')$. En virtud de la simetría de la igualdad matemática, se tiene:

   $$O_i(s') = O_i(s)$$

   lo que implica que $s' \sim_i s$.

   La confusión es recíproca: si la vista de s no permite descartar que el juego esté en s', la vista de s' tampoco permite descartar que esté en s.

3. **Transitividad ($\forall s, s', s'' \in S : (s \sim_i s' \land s' \sim_i s'') \implies s \sim_i s''$):** Sean $s, s', s'' \in S$ tales que $s \sim_i s'$ y $s' \sim_i s''$. Esto significa que:

   $$O_i(s) = O_i(s') \quad \text{y} \quad O_i(s') = O_i(s'')$$

   Por la propiedad transitiva de la igualdad matemática:

   $$O_i(s) = O_i(s'')$$

   lo cual demuestra que $s \sim_i s''$.

   Si s se ve igual que s' y s' se ve igual que s'', los tres estados se ven idénticos para el jugador, y por tanto s y s'' son indistinguibles.

Al satisfacerse la reflexividad, simetría y transitividad, queda formalmente demostrado que $\sim_i$ es una relación de equivalencia sobre S. ■

3.2.4. Conjuntos de información y clases de observación

En un juego en forma extensiva, una historia h identifica una secuencia de acciones y resultados hasta un nodo. Sea H_i el conjunto de historias en las que debe decidir r_i. Un conjunto de información $I_i(h) \subseteq H_i$ agrupa los nodos que ese jugador no distingue a partir de la información que ha recibido, incluido su historial pertinente [32], [33]. No se define, en general, únicamente por la observación del estado presente.

Definición 3.2: Juego de información perfecta

Un juego en forma extensiva es de **información perfecta** cuando cada conjunto de información de cada jugador en sus nodos de decisión es unitario [32], [33]:

$$\forall r_i \in R, \quad \forall h \in H_i : \quad I_i(h) = \{h\}. \tag{3.3}$$

El jugador conoce en qué nodo está al decidir; esto no elimina la incertidumbre sobre las decisiones futuras del adversario ni sobre resultados futuros de azar, si el juego los admite.

Para ilustrar la observación de estados, la relación $\sim_i$ definida antes induce una partición distinta:

$$C_i = S/\sim_i = \{[s]_{\sim_i} \mid s \in S\}. \tag{3.4}$$

Definición 3.3: Clase de estados compatibles con una observación

Para $s \in S$, se denota por $C_i(s)$ la clase de estados que producen la misma observación actual:

$$C_i(s) = [s]_{\sim_i} = \{s' \in S \mid O_i(s') = O_i(s)\}. \tag{3.5}$$

Esta clase describe compatibilidad con una observación de estado; no debe identificarse automáticamente con $I_i(h)$. Historias diferentes pueden llegar a una misma configuración, y un jugador puede conservar información que no aparece en su observación actual.

Proposición 3.2 (Observación completa del estado). *La función $O_i : S \to \Omega_i$ es inyectiva si y solo si $C_i(s) = \{s\}$ para todo $s \in S$.*

Demostración. Si O_i es inyectiva, la igualdad $O_i(s') = O_i(s)$ implica $s' = s$, por lo que la clase contiene únicamente s. Recíprocamente, si todas las clases son unitarias y dos estados producen la misma observación, ambos pertenecen a la misma clase y deben coincidir. ■

La condición de inyectividad se expresa como:

$$\forall s, s' \in S : \quad O_i(s) = O_i(s') \implies s = s'. \tag{3.6}$$

En ese caso, cada observación efectivamente producida identifica un único estado. Para $o \in O_i(S)$ existe una inversa sobre la imagen:

$$s = \left(O_i\big|_{S \to O_i(S)}\right)^{-1}(o). \tag{3.7}$$

Esta inversa no se extiende a observaciones ajenas a la imagen. En el modelo del proyecto se supondrá que el estado y los antecedentes públicos necesarios para aplicar las reglas están disponibles. La observación completa de esa representación se distinguirá de la definición sobre historias de la Ecuación (3.3).

3.2.5. Ejemplos comparativos

Dos ejemplos ayudan a ver la diferencia.

En el Gato ($R = \{\mathrm{x}, \mathrm{o}\}$), que es un juego de información perfecta, el tablero de 3×3 está completamente a la vista de ambos jugadores. Si el estado real es s (por ejemplo, una ficha en el centro), la función de observación de cada participante devuelve exactamente ese tablero: $O_{\mathrm{x}}(s) = s$ y $O_{\mathrm{o}}(s) = s$. Sus clases de observación del estado son, por tanto,

$$C_{\mathrm{x}}(s) = \{s\}, \quad C_{\mathrm{o}}(s) = \{s\} \tag{3.8}$$

Ninguno duda de qué casillas están marcadas; los dos ven toda la realidad del juego a través de un mapeo inyectivo directo. Lo mismo vale para el Ajedrez.

La Batalla Naval, en cambio, es de información imperfecta. Supongamos que el jugador 1 coloca su barco en la coordenada $(1, 1)$ (estado s_A) o en la $(1, 2)$ (estado s_B). Él sabe dónde puso su nave, así que distingue las dos situaciones: $C_1(s_A) = \{s_A\}$ e $C_1(s_B) = \{s_B\}$. El jugador 2, sin embargo, tiene la cuadrícula rival cubierta por niebla, y para él ambos tableros son idénticos: $O_2(s_A) = O_2(s_B) = o$. La función no es inyectiva ($s_A \sim_2 s_B$) y resulta

$$C_2(s_A) = C_2(s_B) = \{s_A, s_B\} \tag{3.9}$$

La clase de observación del jugador 2 tiene dos estados ($|C_2| = 2 > 1$). No sabe si el barco enemigo está en $(1, 1)$ o en $(1, 2)$ y se ve obligado a adivinar o a estimar probabilidades.

La Figura 3.1 ilustra esquemáticamente esta distinción mediante clases de estados compatibles con observaciones inyectivas y no inyectivas. Es una ilustración de observación de estados, no un árbol de conjuntos de información sobre historias.

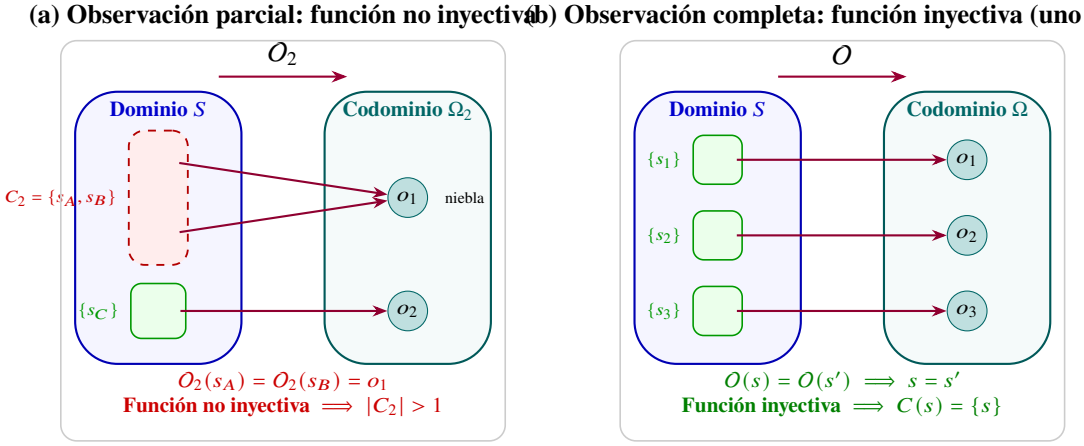


Figura 3.1: Clases de estados compatibles con una observación. En (a), dos estados producen la misma observación y pertenecen a una clase no unitaria; en (b), la observación inyectiva identifica cada estado. La figura ilustra observación de estados, no conjuntos de información sobre historias.

3.2.6. Implicación para el modelo formal Γ

La información perfecta permite razonar sin incertidumbre sobre la historia observada en el nodo de decisión. En el modelo de estados del proyecto se conservarán el turno y los antecedentes necesarios para que las consultas $L_i(s)$, $T(s, \mathbf{a})$ y $G_i(s)$ tengan argumentos suficientes. En

Ajedrez, observar únicamente la colocación de piezas no sustituye los derechos de enroque ni los antecedentes requeridos por las reglas de repetición o captura al paso.

El determinismo de T es un supuesto adicional del alcance, independiente de la Ecuación (3.3). Por ello una acción conjunta legal producirá un único sucesor. El agente seguirá enfrentando incertidumbre estratégica sobre las acciones del rival y, si se estudia MCTS, podrá emplear muestreo aleatorio dentro de un juego de dinámica determinista.

3.3. General Game Playing

Un General Game Player (GGP) es un agente que recibe la descripción formal de un juego y debe jugarlo sin ayuda humana ni conocimiento específico programado de antemano para ese dominio. Genesereth, Love y Pell definen el área como el estudio de sistemas capaces de aceptar la descripción formal de un juego y jugarlo bien sin depender de algoritmos diseñados para juegos particulares [8].

Lo que separa a un motor especializado de un General Game Player es dónde vive el conocimiento del juego. En el motor especializado, las reglas y heurísticas suelen estar dentro del código fuente. En un GGP, las reglas llegan como entrada externa, y el agente debe interpretarlas, construir su propia representación interna y aplicar un proceso de decisión general. Por eso la generalidad no consiste solo en poder ejecutar dos juegos: el mismo núcleo algorítmico tiene que funcionar cuando cambia la descripción del juego.

De aquí se sigue algo para este Trabajo Terminal: no queremos un motor de Ajedrez que rivalice con los sistemas especializados, sino una arquitectura desacoplada. Lo que hay que demostrar es que el sistema carga reglas externas, genera movimientos legales, reconoce estados terminales y escoge jugadas con un criterio general de búsqueda. Y la prueba de que es general está en cambiar el archivo de reglas sin tocar el motor.

3.4. Modelo formal de juego

Para el alcance adoptado, se utilizará un modelo de transición finito y determinista, inspirado en GGP clásico [8], [9], [10]. Las acciones conjuntas ofrecen una representación computacional uniforme; los turnos secuenciales se codificarán con acciones nulas de los roles inactivos. Esta representación no supone que los participantes del TT decidan simultáneamente.

Definición 3.4: Juego en General Game Playing

Un juego formal Γ es un sistema de transición de estados finito, determinista, síncrono y de información perfecta caracterizado por una tupla de ocho componentes [8], [9], [10]:

$$\Gamma = \langle R, S, s_0, \{A_i\}_{i=1}^n, \{L_i\}_{i=1}^n, T, F, \{G_i\}_{i=1}^n \rangle$$

donde:

- $R = \{r_1, r_2, \ldots, r_n\}$ es el conjunto finito de roles o participantes ($|R| = n \geq 1$).
- S es el conjunto finito y no vacío de estados posibles del entorno.

- $s_0 \in S$ es el estado inicial distinguido en el instante $t = 0$.
- A_i es el catálogo finito de acciones primitivas del participante r_i, conformando el espacio conjunto $\mathbf{A} = \prod_{i=1}^{n} A_i$.
- $L_i : S \to \mathcal{P}(A_i)$ es la función de legalidad que asigna a cada estado las acciones permitidas para el participante r_i.
- $T : S \times \mathbf{A} \to S$ es la función de transición dinámica que mapea el estado actual y un vector de acciones conjuntas a un único estado sucesor.
- $F \subseteq S$ es el subconjunto de estados terminales.
- $G_i : S \to [0, 100]$ es la función de utilidad o meta que asigna la recompensa numérica alcanzada por el rol r_i.

A continuación se detalla la semántica y formalización de cada uno de los componentes de la tupla.

3.4.1. Conjunto de roles (R)

El conjunto $R = \{r_1, r_2, \ldots, r_n\}$ es finito y no vacío, con cardinalidad $|R| = n \geq 1$, y sus elementos son los participantes del juego. En *General Game Playing*, esos elementos no tienen para el motor de razonamiento ningún significado previo ("humano", "blancas" o "máquina"); son identificadores abstractos. En el Gato, por ejemplo, $R = \{\text{x}, \text{o}\}$, y en el Ajedrez, $R = \{\text{blancas}, \text{negras}\}$. En GDL cada participante se declara con el predicado `(role r)`.

3.4.2. Espacio de estados (S) y semántica de fluents

El espacio de estados $S = \{s_1, s_2, \ldots, s_k\}$ es un conjunto finito y no vacío con todas las configuraciones admisibles del sistema. En lugar de tratar cada estado como un identificador opaco e indivisible, en *General Game Playing* se descompone en proposiciones estructuradas, a partir del concepto de *fluent* [9], [34].

Definición 3.5: Fluent

Un *fluent* es una proposición lógica atómica (indivisible) $f \in \mathcal{F}$ —concepto originado en el cálculo de situaciones por McCarthy y Hayes [34] y adoptado como base formal en GGP [9], [10]—, cuyo valor de verdad no es estático ni constante, sino una función dependiente del tiempo o del estado del sistema:

$$\text{val}(f, t) \in \{\text{Verdadero}, \text{Falso}\}$$

A diferencia de las relaciones estáticas del dominio, que valen igual durante toda la partida (las dimensiones del tablero o las reglas de adyacencia), un *fluent* representa un hecho contingente del entorno: puede aparecer, mantenerse o desaparecer según las transiciones que provocan las acciones ejecutadas.

Definición 3.6: Estado

En un entorno determinista y de información perfecta, el estado de un juego en el instante t se define lógicamente como la conjunción lógica ($\bigwedge$) de todos los *fluents* atómicos que son verdaderos simultáneamente en ese instante:

$$s_t \equiv \bigwedge_{i=1}^{m} f_i = f_1 \wedge f_2 \wedge \cdots \wedge f_m$$

Bajo la perspectiva conjuntista de la programación lógica declarativa y el principio de mundo cerrado (*Closed World Assumption*) formalizado por Reiter [35], el estado se modela de forma compacta como el subconjunto $s \subseteq \mathcal{F}$ de hechos atómicos verdaderos en dicho instante [9], [10]. En consecuencia, el espacio global de estados satisface:

$$S \subseteq \mathcal{P}(\mathcal{F})$$

donde cualquier *fluent* $f \notin s$ se evalúa formalmente como falso.

Ejemplo ilustrativo en el juego del Gato

Para ver esta semántica en un caso concreto, tomemos el juego del Gato (*Tic-Tac-Toe*) sobre una retícula de 3×3.

El universo de fluents $\mathcal{F}$ tiene dos clases de proposiciones atómicas. Las primeras describen la ocupación de cada casilla: $\mathcal{F}_{\text{celdas}} = \{\text{cell}(r, c, m) \mid r \in \{1, 2, 3\},\ c \in \{1, 2, 3\},\ m \in \{\text{x}, \text{o}, \text{b}\}\}$, donde b indica una casilla vacía (*blank*); son $3 \times 3 \times 3 = 27$ átomos posibles. Las segundas indican a quién le toca jugar: $\mathcal{F}_{\text{turno}} = \{\text{control}(\text{x}), \text{control}(\text{o})\}$, que son 2 átomos. Así, $\mathcal{F} = \mathcal{F}_{\text{celdas}} \cup \mathcal{F}_{\text{turno}}$, con $|\mathcal{F}| = 29$.

Al empezar la partida, las nueve casillas están vacías y el turno es de x. El estado inicial s_0 es entonces la conjunción de diez fluents verdaderos:

$$s_0 \equiv \left(\bigwedge_{r=1}^{3} \bigwedge_{c=1}^{3} \text{cell}(r, c, \text{b}) \right) \wedge \text{control}(\text{x}) \tag{3.10}$$

o, en su representación como conjunto:

$$s_0 = \{\text{cell}(1, 1, \text{b}), \text{cell}(1, 2, \text{b}), \ldots, \text{cell}(3, 3, \text{b}), \text{control}(\text{x})\} \tag{3.11}$$

Proposiciones como $\text{cell}(2, 2, \text{x})$ no pertenecen a s_0 y, por lo tanto, son falsas en $t = 0$.

Si x marca el centro con la acción $\text{mark}(2, 2)$, la función de transición quita los fluents invalidados y agrega los nuevos:

$$s_1 = \big(s_0 \setminus \{\text{cell}(2, 2, \text{b}), \text{control}(\text{x})\}\big) \cup \{\text{cell}(2, 2, \text{x}), \text{control}(\text{o})\} \tag{3.12}$$

Como conjunción, la nueva configuración s_1 queda así:

$$s_1 \equiv \text{cell}(2, 2, \text{x}) \wedge \text{control}(\text{o}) \wedge \bigwedge_{(r,c) \neq (2,2)} \text{cell}(r, c, \text{b}) \tag{3.13}$$

Las ocho celdas vacías restantes se mantienen igual por inercia.

En GDL, para preguntar si un *fluent* vale en el estado actual se usa `(true f)`. De cómo empieza el juego y de cómo cambia se ocupan `(init f)` y `(next f)`; más adelante, en los listados del Gato, se ven los tres en acción [9], [10].

3.4.3. Estado inicial (s_0)

El elemento distinguido $s_0 \in S$ es la configuración del entorno en el instante $t = 0$, antes de que los agentes tomen ninguna decisión. En una descripción declarativa, s_0 se construye con todos los hechos que se declaran mediante el predicado `(init f)`.

3.4.4. Acciones individuales (A_i) y espacio conjunto de acciones (A)

Para cada rol $r_i \in R$, el conjunto finito $A_i = \{a_{i,1}, a_{i,2}, \ldots, a_{i,m_i}\}$ reúne las acciones primitivas que ese participante puede concebir. Como el modelo es síncrono, en cada paso de tiempo todos los jugadores eligen una acción a la vez, y el espacio global de decisiones es el producto cartesiano de los conjuntos individuales:

$$\mathbf{A} = \prod_{i=1}^{n} A_i = A_1 \times A_2 \times \cdots \times A_n \tag{3.14}$$

Un elemento $\mathbf{a} = (a_1, a_2, \ldots, a_n) \in \mathbf{A}$ es una acción conjunta, un vector de decisiones concurrentes en el que cada componente $a_i \in A_i$ es la jugada del rol r_i.

3.4.5. Función de legalidad (L_i)

Que una acción sea válida depende del estado actual del sistema. La legalidad de cada jugador r_i se modela con la correspondencia:

$$L_i : S \to \mathcal{P}(A_i) \tag{3.15}$$

que asigna a cada configuración $s \in S$ el subconjunto de acciones permitidas $L_i(s) \subseteq A_i$. En su formulación relacional original [8], [9], el criterio se expresa como una relación binaria $L_i \subseteq A_i \times S$, en la que $(a, s) \in L_i$ significa que la acción a se puede ejecutar en s; corresponde directamente al predicado `(legal r a)` [10].

Una acción conjunta $\mathbf{a} = (a_1, \ldots, a_n)$ es legal en el estado s si y solo si cada componente es legal para su rol:

$$\mathbf{a} \in \mathbf{L}(s) \iff \forall i \in \{1, \ldots, n\}, \quad a_i \in L_i(s) \tag{3.16}$$

donde $\mathbf{L}(s) = \prod_{i=1}^{n} L_i(s)$ es el espacio de jugadas conjuntas válidas en ese estado.

3.4.6. Función de transición dinámica (T)

La evolución del juego en el tiempo la rige la función de transición determinista:

$$T : S \times \mathbf{A} \to S \tag{3.17}$$

que determina de forma única el estado sucesor $s' = T(s, \mathbf{a})$ que resulta de aplicar la acción conjunta $\mathbf{a} \in \mathbf{L}(s)$ sobre la configuración s. Dos propiedades la caracterizan. Es estrictamente determinista: a diferencia de los procesos estocásticos o del control continuo, evaluar las reglas produce un único estado resultante y no hay incertidumbre en la física del sistema. Y es reactiva y síncrona: el universo cambia solo como consecuencia de las decisiones conjuntas de los agentes, algo que en GDL se especifica con las reglas de deducción de hechos futuros `(next f)`.

3.4.7. Conjunto de estados terminales (F)

El subconjunto $F \subseteq S$ marca las condiciones en las que termina una partida. Cuando el sistema llega a una configuración $s \in F$, la trayectoria se detiene: ya no se admiten más movimientos y se calculan los resultados finales. En los juegos de mesa, este conjunto incluye situaciones como el jaque mate, las tablas por repetición, una línea ganadora o el tablero lleno. En el motor lógico se verifica con el predicado `(terminal)`.

3.4.8. Función de utilidad o meta (G_i)

Para cada participante $r_i \in R$, la función de utilidad o pago:

$$G_i : S \to [0, 100] \tag{3.18}$$

Aunque formalmente se puede evaluar en cualquier estado, la función tiene sentido sobre todo en los estados terminales $s \in F$. En GDL clásico los valores de meta son enteros entre 0 y 100 [10]. Para los ejemplos de Gato y Ajedrez se adoptará la convención 100 para victoria, 0 para derrota y 50 para empate; la interpretación de esos valores deberá declararse en cada juego. En este modelo se consultará la utilidad como resultado de la partida en los estados terminales; no se confundirá con una evaluación heurística de estados no terminales.

En los juegos adversariales de dos participantes y suma constante, como el Ajedrez y el Gato, las utilidades terminales cumplen:

$$G_1(s) + G_2(s) = 100, \quad \forall s \in F \tag{3.19}$$

lo cual asegura el carácter competitivo de suma constante que exigen algoritmos como Minimax. Esta función se calcula con la relación lógica `(goal r v)`.

3.4.9. Axiomas de admisibilidad y juegos bien formados

Para que un agente autónomo pueda resolver computacionalmente un juego formalizado como máquina de estados, se piden dos propiedades estructurales [8], [9]:

Definición 3.7: Juego bien formado

Un juego formal Γ se define como bien formado (*well-formed game*) [8], [9], [10] si satisface dos axiomas fundamentales:

1. **Jugabilidad (*Playability*):** Para todo estado no terminal, cada rol dispone de al menos

una acción legal ejecutable:

$$\forall s \in S \setminus F, \quad \forall r_i \in R : \quad L_i(s) \neq \emptyset$$

Esta condición previene bloqueos indeterminados (*deadlocks*) en la ejecución algorítmica del motor.

2. **Terminación garantizada (*Termination*):** Toda trayectoria legal inducida a partir del estado inicial s_0:

$$s_0 \xrightarrow{\mathbf{a}_0} s_1 \xrightarrow{\mathbf{a}_1} s_2 \cdots \xrightarrow{\mathbf{a}_{k-1}} s_k$$

debe culminar forzosamente en un estado terminal $s_k \in F$ en un número finito de transiciones ($k < \infty$).

En dominios donde podrían presentarse ciclos infinitos, la terminación se asegura con reglas de repetición o con cotas explícitas de turnos.

3.4.10. Distancia en el grafo de estados y estimación hacia configuraciones terminales

El modelo Γ induce un grafo dirigido: sus vértices son estados y sus aristas son transiciones producidas por acciones conjuntas legales desde estados no terminales. Una trayectoria expresa una sucesión posible de jugadas; por sí sola no representa una estrategia que garantice un resultado ante cualquier respuesta del adversario.

Definición 3.8: Distancia dirigida entre estados

Para $s_a, s_b \in S$, se define $d(s_a, s_b)$ como la longitud mínima de una trayectoria legal desde s_a hasta s_b [1], [36]:

$$d(s_a, s_b) = \min \left\{ k \in \mathbb{N}_0 \;\middle|\; \begin{array}{l} \exists \mathbf{a}_0, \ldots, \mathbf{a}_{k-1} : \\ s^{(0)} = s_a,\ s^{(k)} = s_b, \\ \forall\, 0 \leq j < k :\ s^{(j)} \notin F,\ \mathbf{a}_j \in \mathbf{L}(s^{(j)}), \\ s^{(j+1)} = T(s^{(j)}, \mathbf{a}_j) \end{array} \right\}. \tag{3.20}$$

Se adopta $\min \emptyset = \infty$. La trayectoria vacía admite $k = 0$, de modo que $d(s, s) = 0$. No es una métrica ordinaria: puede ser asimétrica y puede valer infinito. El superíndice $s^{(j)}$ indica el paso de la trayectoria, no una potencia.

Para un conjunto de destinos $B \subseteq S$, se entenderá $d(s, B) = \min_{b \in B} d(s, b)$, también con valor infinito si no existe destino alcanzable. En particular:

$$d(s, F) = \min_{s_f \in F} d(s, s_f). \tag{3.21}$$

Si el juego finito satisface el axioma de terminación para toda trayectoria legal alcanzable desde

s_0, existe un horizonte máximo finito $H_{\text{máx}}$ y:

$$\forall s \in \text{Reach}(s_0): \quad 0 \leq d(s, F) \leq H_{\text{máx}} < \infty. \tag{3.22}$$

Además, $d(s, F) = 0$ si y solo si $s \in F$. La cota depende del supuesto de terminación; la finitud del conjunto de estados no excluye por sí sola ciclos infinitos.

Con la convención de utilidad de los ejemplos y desde el rol r_i, sean $F_{\text{victoria}} = \{s \in F \mid G_i(s) = 100\}$ y $F_{\text{derrota}} = \{s \in F \mid G_i(s) = 0\}$. Las distancias son:

$$d(s, F_{\text{victoria}}) = \min_{s_f \in F_{\text{victoria}}} d(s, s_f), \tag{3.23}$$

$$d(s, F_{\text{derrota}}) = \min_{s_f \in F_{\text{derrota}}} d(s, s_f). \tag{3.24}$$

Puede existir una trayectoria corta hacia la victoria que requiera un error del oponente. Por ello, minimizar la primera distancia no equivale a asegurar una victoria. Minimax optimiza la utilidad que puede garantizar frente a respuestas adversariales, según la recurrencia por turnos de la Ecuación (3.32). Un desempate por duración requeriría definir su tratamiento sobre estrategias y no solo sobre el camino favorable más corto.

Cuando no se alcanza la frontera terminal se utiliza una heurística. La cercanía a una meta puede ser un rasgo de esa evaluación, pero no es por definición una probabilidad de ganar ni garantiza utilidad alta.

Una cota ilustrativa se obtiene para una meta conjuntiva $\mathcal{M} \subseteq \mathcal{F}$. Sea $F_{\mathcal{M}} = \{s_f \in F \mid \mathcal{M} \subseteq s_f\}$ y:

$$\text{Faltantes}(s, \mathcal{M}) = \{f \in \mathcal{M} \mid f \notin s\}. \tag{3.25}$$

Si en cualquier transición legal se pueden incorporar como máximo $c \geq 1$ hechos de esa meta que antes eran falsos, entonces:

$$d(s, F_{\mathcal{M}}) \geq \left\lceil \frac{|\,\text{Faltantes}(s, \mathcal{M})|}{c} \right\rceil. \tag{3.26}$$

La desigualdad cuenta transiciones conjuntas. En Gato, para una línea fija de X, cada transición agrega a lo sumo una marca de X ($c = 1$). Si falta una marca, se requiere al menos una transición; si es turno de O o este bloquea la línea, la cota puede ser poco informativa o el destino inalcanzable. No implica una victoria forzada ni se extiende sin justificar sus supuestos a cualquier condición terminal.

En planificación, la **relajación de efectos negativos** elimina las listas de borrado de los operadores: los hechos añadidos no se destruyen [36], [37]. Para tareas proposicionales STRIPS finitas y explícitas, comprobar alcanzabilidad en esa relajación es polinomial, pero obtener la longitud del plan relajado óptimo sigue siendo NP-hard. El óptimo relajado proporciona una cota inferior del costo original bajo esa relajación; aproximaciones eficientes como la extracción de planes de FF no garantizan en general admisibilidad [37]. Su traslado a reglas de juegos requiere justificar la representación, la relajación y el papel del adversario; no se considera una garantía general de GGP ni un algoritmo seleccionado para este proyecto.

3.4.11. Modelado síncrono de juegos por turnos alternados

Aunque $\mathbf{A} = A_1 \times A_2$ reúne las acciones de ambos participantes, los juegos típicamente secuenciales, como el Gato o el Ajedrez, encajan en este esquema mediante una acción pasiva o nula, noop (*no-operation*) [9], [10].

En un estado s donde le corresponde mover al jugador blanco, la función de legalidad asigna:

$$L_{\text{blancas}}(s) \subseteq A_{\text{blancas}} \setminus \{\text{noop}\} \tag{3.27}$$

$$L_{\text{negras}}(s) = \{\text{noop}\} \tag{3.28}$$

y cada acción conjunta admisible tiene la forma:

$$\mathbf{a} = (a_{\text{blanca}}, \text{noop}) \tag{3.29}$$

Al calcular la transición $s' = T(s, \mathbf{a})$ mediante la función de la Ecuación (3.17), las reglas del juego actualizan los fluents de control y si el sucesor no es terminal, se invierte la legalidad en la siguiente configuración:

$$L_{\text{blancas}}(s') = \{\text{noop}\} \tag{3.30}$$

$$L_{\text{negras}}(s') \subseteq A_{\text{negras}} \setminus \{\text{noop}\} \tag{3.31}$$

Así la máquina de estados conserva una arquitectura síncrona homogénea sin perder la alternancia secuencial propia del dominio.

3.4.12. Interfaz computacional para el motor de decisión

Para que esta formalización pueda usarse en la arquitectura de software del agente, el motor de juego debe ofrecer una interfaz programática con cinco consultas básicas: obtener el estado inicial (s_0); consultar las acciones legales de un jugador en un estado dado ($L_i(s)$); aplicar una acción conjunta y calcular el estado sucesor ($T(s, \mathbf{a})$); determinar si un estado cumple la condición de terminación ($s \in F$); y evaluar la utilidad que obtiene cada participante ($G_i(s)$).

Con esta interfaz, el motor de búsqueda queda completamente separado de las particularidades de cada juego. El algoritmo de decisión razona solo sobre estados abstractos, acciones legales y recompensas numéricas, sin conocimiento empírico propio del dominio.

3.5. Lenguaje de descripción de juegos (GDL)

Para que un agente pueda jugar sin que nadie le explique las reglas, estas tienen que estar escritas en un lenguaje que él mismo pueda leer, analizar y ejecutar. Eso es un lenguaje de descripción de juegos. En *General Game Playing*, el de referencia es el *Game Description Language* (GDL), desarrollado en la Universidad de Stanford [8], [10].

En un lenguaje imperativo como C++ o Python, la lógica de un juego se programa con estructuras de control, bucles y modificaciones destructivas de la memoria: se ordena qué hacer paso a paso. Un lenguaje de descripción de reglas va por otro lado, el declarativo, y en lugar de decir cómo se ejecuta el juego, dice qué es.

3.5.1. Características fundamentales de GDL

A partir de los fundamentos de Genesereth et al. [8], Love et al. [10] y Cropper et al. [38], pueden destacarse cuatro propiedades de GDL.

La primera es que se basa en lógica de primer orden y es declarativo. El autor del juego no programa cómo se ejecutan las jugadas: solo declara qué hechos son verdaderos y bajo qué condiciones cambian, y es el motor de inferencia del agente el que deduce las consecuencias de esas reglas. La segunda es su independencia del dominio. GDL no trae ideas previas sobre geometría de tableros, piezas de ajedrez o cartas; todo se representa con símbolos abstractos, y un agente general aprende la física y las restricciones del entorno mientras procesa el archivo de reglas, porque no las conoce de antemano.

La tercera es su semántica, basada en Datalog con negación estratificada. GDL es una variante estricta de Datalog que limita el uso de funciones recursivas complejas, lo que garantiza que el espacio de estados y la deducción lógica sean finitos y decidibles en cada paso de tiempo, y evita ciclos de inferencia infinitos. La cuarta es el desacoplamiento arquitectónico: la física del juego, descrita en el archivo de reglas, queda totalmente separada de los algoritmos de búsqueda y decisión, que viven en el motor del agente. Por eso el mismo núcleo puede jugar juegos muy distintos sin cambiar una línea de código fuente.

3.5.2. Concepto y estructura de un predicado lógico

Para entender la sintaxis de GDL sin caer en un formalismo matemático excesivo, conviene partir de la experiencia del desarrollo de software:

Definición 3.9: Predicado lógico

En la semántica lógica declarativa de GDL [9], [10], un **predicado** es una afirmación estructurada sobre uno o varios objetos del dominio que, al ser evaluada contra el estado actual del juego, devuelve estrictamente un valor de verdad: verdadero o falso.

Un programador que quiere saber si hay una cruz en el renglón 1 y la columna 1 escribiría algo como `tablero[1][1] == 'x'`: mirar una celda de una matriz. GDL no consulta nada, afirma un hecho. La misma pregunta se escribe

```
(cell 1 1 x)
```

y la respuesta es simplemente si ese hecho es verdadero o falso en el estado actual.

Un predicado se escribe como una lista entre paréntesis y tiene tres partes. La primera es el *functor*, el identificador inicial que dice qué propiedad o relación se expresa (por ejemplo, `cell`, `control`, `legal`, `next`). La segunda son los términos o argumentos, los elementos sobre los que recae la afirmación. Estos pueden ser constantes, símbolos atómicos que denotan objetos concretos del dominio, como coordenadas (`1`, `2`, `3`) o identificadores (`x`, `o`, y `b` para una casilla vacía o *blank*); o variables, que en GDL son cualquier símbolo que empieza con un signo de interrogación (`?r` para el renglón, `?c` para la columna, `?p` para el participante) y permiten escribir reglas generales válidas para cualquier casilla o jugador.

La tercera parte es la aridad, el número de argumentos que acepta el predicado. Con aridad 0 tenemos una proposición pura, como `(terminal)`, que no recibe argumentos y expresa un hecho

global: si la partida terminó. Con aridad 1, una propiedad de un objeto, por ejemplo `(role x)` o `(control x)`, que indican que `x` es un rol o que tiene el turno. Con aridad 3, una relación ternaria como `(cell 2 2 x)`, que une renglón, columna y ficha.

Construcción de reglas de inferencia (<=)

Los hechos no siempre están dados de forma estática; muchas veces hay que deducirlos de otros hechos. Para eso, GDL usa reglas de inferencia con el operador de implicación <=, que se lee "la cabeza es verdadera si todas las premisas del cuerpo son verdaderas":

```
(<= (cabeza_deducida)
    (premisa_o_condicion_1)
    (premisa_o_condicion_2)
    ...
    (premisa_o_condicion_k))
```

Código 3.1: Estructura canónica de una regla de inferencia en GDL.

El motor lógico solo da por cierta la cabeza si logra verificar que todas las condiciones del cuerpo se cumplen a la vez en el estado del juego.

3.5.3. Predicados reservados del sistema en GDL

Para que un archivo de reglas describa la tupla formal Γ de la Sección 3.4, GDL define un conjunto cerrado de ocho predicados del sistema [9], [10], que se resumen en la Tabla 3.1.

Tabla 3.1: Predicados reservados de GDL y componente de Γ que describen.

Predicado	Función
`(role ?r)`	Declara a los participantes del juego e instancia el conjunto R.
`(init ?f)`	Especifica los fluents atómicos que forman el estado inicial s_0.
`(true ?f)`	Consulta si un fluent dinámico f es verdadero en el estado actual s.
`(does ?r ?a)`	Representa la acción $a \in A_i$ que el rol r ejecuta en el turno corriente.
`(legal ?r ?a)`	Deduce si la acción a pertenece al conjunto de jugadas permitidas $L_i(s)$.
`(next ?f)`	Deduce qué fluents formarán parte del estado sucesor $s' = T(s, \mathbf{a})$.
`(terminal)`	Evalúa si la configuración actual pertenece al conjunto de estados terminales F.
`(goal ?r ?v)`	Determina la utilidad o recompensa numérica $G_i(s) = v \in [0, 100]$ del jugador r.

3.5.4. Ejemplificación mediante código de especificación: El juego del Gato

Para ver cómo trabajan juntos estos predicados, tomemos la especificación del juego del Gato (*Tic-Tac-Toe*), que se puede dividir en cuatro bloques lógicos.

Roles y estado inicial

El primer bloque (Código 3.2) declara los dos jugadores y describe el tablero al inicio ($t = 0$): las nueve casillas vacías (`b`) y el turno en manos de `x`.

```
;; ========================================
;; 1. Declaracion de roles (Jugadores)
;; ========================================
(role x)
(role o)

;; ========================================
;; 2. Estado inicial (t = 0):
;;    Nueve casillas vacias (b = blank)
;;    y control asignado a x
;; ========================================
(init (cell 1 1 b))
(init (cell 1 2 b))
(init (cell 1 3 b))
(init (cell 2 1 b))
(init (cell 2 2 b))
(init (cell 2 3 b))
(init (cell 3 1 b))
(init (cell 3 2 b))
(init (cell 3 3 b))
(init (control x))
```

Código 3.2: Declaración de roles y estado inicial para el juego del Gato.

Reglas de legalidad y alternancia de turnos

El segundo bloque (Código 3.3) formaliza la función de legalidad $L_i(s)$. Una jugada es legal si quien la intenta tiene el control y la casilla elegida está libre. La sincronía se modela con la jugada nula noop:

```
;; ========================================
;; 3. Acciones legales
;; ========================================
;; Un jugador puede marcar (?r, ?c) si es su turno
;; y la casilla actualmente contiene 'b'
(<= (legal ?p (mark ?r ?c))
    (true (control ?p))
    (true (cell ?r ?c b)))

;; Si NO es el turno del jugador, su unica accion legal es 'noop'
(<= (legal ?p noop)
    (role ?p)
    (not (true (control ?p))))
```

Código 3.3: Reglas de legalidad de movimientos en GDL.

Con estas reglas el motor rechaza por sí solo cualquier intento de poner una ficha en una casilla ocupada, y al rival que no tiene el turno le deja únicamente la acción nula.

Transición de estado y persistencia inercial

El tercer bloque (Código 3.4) gobierna la función de transición $T(s, \mathbf{a})$. En lógica declarativa, un hecho no se conserva en el estado siguiente a menos que una regla lo diga de forma explícita, así que hay que definir tanto los cambios directos como la inercia del sistema:

```
;; ========================================
;; 4. Funcion de transicion (Siguiente estado)
;; ========================================
;; A. Efecto directo: la casilla marcada pasa a tener la ficha del jugador
(<= (next (cell ?r ?c ?p))
    (does ?p (mark ?r ?c)))

;; B. Inercia de fichas: las celdas ya marcadas conservan su contenido
(<= (next (cell ?r ?c ?mark))
    (true (cell ?r ?c ?mark))
    (distinct ?mark b))

;; C. Inercia de celdas vacias: una casilla sigue vacia si nadie la marco
(<= (next (cell ?r ?c b))
    (true (cell ?r ?c b))
    (not (does x (mark ?r ?c)))
    (not (does o (mark ?r ?c))))

;; D. Alternancia del turno
(<= (next (control o)) (true (control x)))
(<= (next (control x)) (true (control o)))
```

Código 3.4: Reglas de transición dinámica e inercia en GDL.

Condiciones terminales y asignación de recompensas

El cuarto bloque (Código 3.5) define el conjunto de estados terminales F y la función de utilidad $G_i(s)$: premia la formación de líneas de tres fichas y reconoce el empate cuando el tablero se llena:

Los cuatro bloques incluyen los auxiliares `line`, `open` y `board_full`. Las líneas se restringen a los roles para que las casillas vacías no se interpreten como una victoria. La partida se detendrá al deducir `terminal`; las consultas de legalidad y transición se utilizarán únicamente mientras el estado actual no sea terminal. Es una especificación didáctica de Gato en GDL, no una prueba de que el intérprete del proyecto ya esté implementado ni una elección del lenguaje definitivo.

3.6. GDL, Ludii y lenguajes de dominio específico

Describir un juego con GDL clásico es una opción entre varias. Otros lenguajes intentan que la escritura sea más cómoda sin perder expresividad ni volverse difíciles de implementar. Ludii es un caso claro: en vez de predicados lógicos habla de tableros, piezas y movimientos, más cerca de cómo piensa quien diseña juegos [28].

```
;; ========================================
;; 5. Lineas, tablero lleno y utilidades
;; ========================================
;; Lineas completas: filas, columnas y las dos diagonales
(<= (line ?p)
    (role ?p)
    (true (cell ?r 1 ?p))
    (true (cell ?r 2 ?p))
    (true (cell ?r 3 ?p)))
(<= (line ?p)
    (role ?p)
    (true (cell 1 ?c ?p))
    (true (cell 2 ?c ?p))
    (true (cell 3 ?c ?p)))
(<= (line ?p)
    (role ?p)
    (true (cell 1 1 ?p))
    (true (cell 2 2 ?p))
    (true (cell 3 3 ?p)))
(<= (line ?p)
    (role ?p)
    (true (cell 1 3 ?p))
    (true (cell 2 2 ?p))
    (true (cell 3 1 ?p)))

;; La negacion se aplica a un predicado sin variables libres
(<= open (true (cell ?r ?c b)))
(<= board_full (not open))

;; Victoria absoluta (100 puntos) si el jugador forma una linea
(<= (goal ?p 100) (line ?p))

;; Derrota (0 puntos) si el adversario completo una linea
(<= (goal ?p 0)   (role ?p) (line ?other) (distinct ?p ?other))

;; Empate (50 puntos) si el tablero esta lleno y nadie gano
(<= (goal ?p 50)  (role ?p) (not (line x)) (not (line o)) board_full)

;; ========================================
;; 6. Condicion de fin de partida (terminal)
;; ========================================
(<= terminal (line x))
(<= terminal (line o))
(<= terminal board_full)
```

Código 3.5: Reglas de utilidad (goal) y condición de terminación en GDL.

Comparar GDL con Ludii importa para este Trabajo Terminal porque obliga a tomar una decisión de arquitectura: implementar GDL completo, adoptar solo un subconjunto o diseñar un lenguaje de dominio específico (DSL) inspirado en ambos. GDL tiene una base académica sólida y una semántica lógica general, pero puede resultar pesado para un prototipo de alcance acotado. Ludii, por su parte, demuestra que un lenguaje puede seguir siendo general y ser más legible para quienes diseñan y desarrollan juegos.

Un DSL propio permitiría concentrarse en lo que el proyecto necesita: jugadores, estados, acciones, reglas de legalidad, transiciones, condiciones terminales y recompensas. Esta opción tendrá que justificarse formalmente, para que el lenguaje no termine siendo una colección de casos especiales. El criterio de diseño es que permita describir los juegos dentro del alcance definido: discretos, deterministas, por turnos y de información perfecta.

3.7. Intérprete y representación interna

Tener el archivo de reglas no basta para jugar. Hace falta un intérprete que lea la descripción, valide su estructura y construya una representación interna que el motor del agente pueda consultar. El proceso suele pasar por análisis léxico, análisis sintáctico, validación semántica y construcción de un modelo interno.

El análisis léxico separa el texto en unidades reconocibles y el sintáctico comprueba que esas unidades respeten la gramática del lenguaje. La validación semántica revisa que las reglas tengan sentido dentro del modelo de juego; por ejemplo, que las acciones hagan referencia a jugadores que existen o que una condición terminal pueda evaluarse. Al final, la representación interna organiza toda esa información para responder consultas durante la partida.

La representación interna hace de puente entre el lenguaje textual y los algoritmos de decisión. Una vez construido el modelo, el motor ya no debería trabajar con cadenas de texto, sino con estructuras que permitan evaluar legalidad, transiciones y recompensas. Esto ayuda al mantenimiento: en principio se podría cambiar el lenguaje textual sin rehacer el motor de búsqueda, siempre que se conserve la interfaz conceptual del juego.

En las implementaciones avanzadas de GGP, las reglas también pueden compilarse en estructuras más eficientes. Goldwaser y Thielscher señalan que una descripción GDL puede convertirse en una red proposicional (PropNet), con nodos lógicos, de entrada, de salida y de transición [31]. Esto interesa porque interpretar las reglas desde cero en cada nodo de búsqueda resulta caro. Una PropNet completa quizá quede fuera de una primera versión, pero la idea sirve para entender por qué la eficiencia de la representación interna pesa tanto como el algoritmo de búsqueda.

3.8. Árboles de juego y explosión combinatoria

Con acciones legales y transiciones ya se puede armar un árbol de juego [1]: la raíz es el estado actual, de ella salen tantas ramas como acciones legales haya, y cada rama termina en el estado que produce esa acción. Si el proceso se repite, el agente ve hacia adelante y puede comparar a dónde lleva cada decisión.

El problema es que el número de nodos crece muy rápido. Si un estado tiene un factor de ramificación promedio de b acciones y se explora hasta la profundidad d, el número de nodos puede crecer del orden de b^d. En juegos pequeños es manejable; en juegos complejos, explorarlo todo no es práctico.

Por la explosión combinatoria hay que limitar la búsqueda: el agente debe decidir cuánto explorar, qué ramas priorizar y cómo evaluar los estados que no son terminales. Aquí entran Minimax, la poda alfa-beta y Monte Carlo Tree Search, junto con funciones heurísticas que estiman qué tan bueno es un estado sin llegar hasta el final de la partida.

3.9. Minimax

Minimax es un algoritmo clásico para juegos adversariales de dos jugadores. Se apoya en el teorema minimax de von Neumann [39] y fue formulado para decidir en árboles de juego por Shannon [12] y, más tarde, por Russell y Norvig [1]. La idea básica es suponer que un jugador trata de maximizar su utilidad mientras que el oponente trata de minimizarla, de modo que en el árbol los niveles se alternan entre decisiones de MAX y de MIN.

Definición 3.10: Función de valor Minimax y decisión óptima

En un juego formal de dos participantes ($R = \{r_1, r_2\}$) determinista, de información perfecta y suma constante [1], [12], [39], el **valor Minimax** $V(s)$ de un estado $s \in S$ desde la perspectiva de MAX (r_1) se define recursivamente como:

$$V(s) = \begin{cases} G_1(s) & \text{si } s \in F \\ \max_{a \in L_1(s)} V(T(s, (a, \text{noop}))) & \text{si el turno corresponde a } r_1 \\ \min_{a \in L_2(s)} V(T(s, (\text{noop}, a))) & \text{si el turno corresponde a } r_2 \end{cases} \tag{3.32}$$

La **decisión óptima** $a^* \in L_i(s)$ para el participante activo corresponde a la acción que optimiza (maximiza para r_1, minimiza para r_2) el valor del estado sucesor inducido.

Si el árbol puede explorarse por completo hasta los estados terminales F, Minimax garantiza elegir una acción óptima siempre que ambos jugadores actúen racionalmente. El valor de un estado terminal sale directo de la función de utilidad $G_1(s)$, y el de cualquier estado no terminal se calcula propagando hacia arriba los valores de sus sucesores: MAX toma el mayor y MIN el menor.

En la práctica, casi todos los juegos tienen factores de ramificación demasiado grandes para llegar a la frontera terminal. Por eso se fija una profundidad máxima de búsqueda y se evalúan los estados de la frontera con una función heurística. Así Minimax se puede usar en juegos más grandes, pero aparece la pregunta central para *General Game Playing*: cómo evaluar estados sin usar conocimiento del dominio programado de antemano.

3.10. Poda alfa-beta

La poda alfa-beta es una optimización básica de Minimax, analizada con detalle por Knuth y Moore [40] y descrita en la literatura clásica de inteligencia artificial [1]. Bajo las mismas condiciones de evaluación no cambia la decisión final, pero evita explorar las ramas que no pueden modificar el resultado. Para ello mantiene dos valores: alfa, la mejor alternativa que ha encontrado MAX, y beta, la mejor que ha encontrado MIN.

Cuando detecta que una rama no puede dar una decisión mejor que otra ya conocida, deja de explorarla. Así visita menos nodos y puede llegar a mayor profundidad con el mismo presupuesto de cómputo. Su efectividad depende del orden en que se exploren las acciones: si las mejores se revisan temprano, se descartan más ramas.

En este Trabajo Terminal, alfa-beta interesa porque permite comparar dos cosas: la decisión que produce Minimax y la eficiencia que se gana al podar. En la evaluación experimental, métricas como los nodos explorados, el tiempo de decisión y la profundidad alcanzada pueden mostrar si la optimización mejora el rendimiento sin afectar la legalidad ni la coherencia de las jugadas. La Figura 3.2 muestra cómo funciona el corte: en cuanto MIN encuentra una alternativa con valor menor o igual a la cota α que MAX ya tiene asegurada, se descartan de inmediato los demás sucesores de ese subárbol.

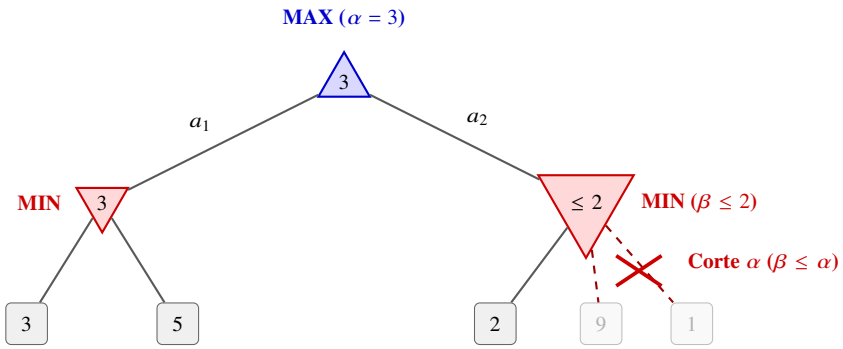


Figura 3.2: Árbol de juego adversarial con poda alfa-beta. Tras evaluarse el nodo con valor 2, el nodo MIN asegura un valor a lo sumo de 2, el cual es estrictamente inferior al valor $\alpha = 3$ ya garantizado por MAX en la rama izquierda; en consecuencia, se interrumpe la exploración del resto de los sucesores.

3.11. Funciones heurísticas generales

Una función heurística estima qué tan favorable es un estado cuando no se puede explorar el árbol hasta el final. En los motores especializados suele llevar conocimiento del dominio; en Ajedrez, por ejemplo, se puede dar un valor a cada pieza o medir el control del centro. Pero esas reglas no son generales: si el agente sabe cuánto vale una reina, ya trae conocimiento específico de Ajedrez.

En GGP el problema es evaluar sin saber de qué juego se trata. Se puede recurrir a rasgos estructurales: cuántas acciones legales hay (la movilidad), qué tan cerca se está de un estado terminal, cómo cambian las recompensas o qué resultados dan las simulaciones. Ninguno de estos

rasgos funciona bien en todos los juegos; lo valioso es que ninguno presupone un dominio y, combinados, pueden aproximar una evaluación en un agente que no conoce el juego.

Aquí está uno de los puntos más delicados del proyecto. Limitar la búsqueda obliga a decidir qué estados parecen mejores, pero ser generales impide escribir esa evaluación para un juego concreto. De ahí que las heurísticas generales y los métodos basados en simulación ocupen un lugar central en la fundamentación teórica del motor de decisión.

3.12. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) es una familia de métodos de búsqueda que estima el valor de las acciones mediante simulaciones. En vez de depender solo de una función heurística explícita, va construyendo poco a poco un árbol de búsqueda y usa los resultados de partidas simuladas para orientar la exploración [23], [41].

MCTS funciona como un ciclo iterativo y asimétrico de cuatro fases que se repite mientras haya tiempo de cómputo, como se ve en la Figura 3.3. En la *selección*, el algoritmo baja por el árbol acumulado desde la raíz eligiendo los sucesores más prometedores con una política de balance. En la *expansión* se agregan uno o varios nodos hijos que corresponden a acciones legales aún no exploradas. En la *simulación*, o *rollout*, se juega una partida rápida con movimientos muestreados al azar hasta llegar a una configuración terminal. Por último, en la *retropropagación*, la recompensa terminal se propaga hacia la raíz y actualiza las estadísticas de visitas y el valor medio de cada estado del camino recorrido.

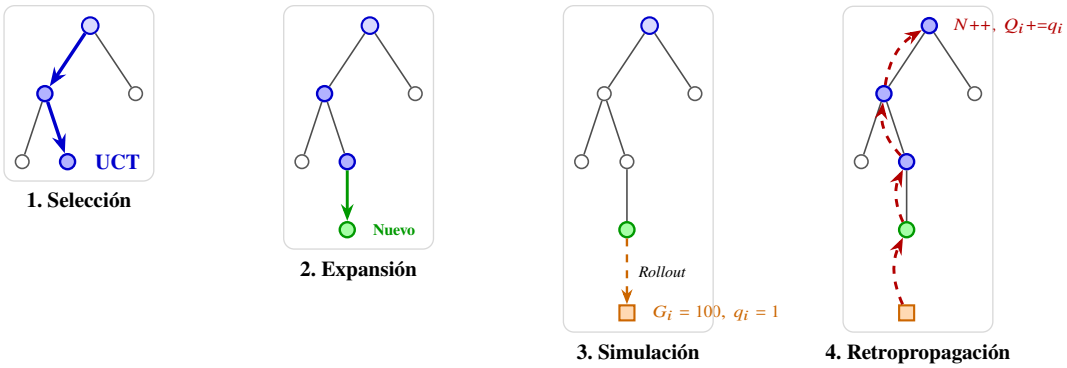


Figura 3.3: Ciclo iterativo de cuatro fases en Monte Carlo Tree Search (MCTS): Selección de nodos mediante la cota UCT, Expansión de nuevas acciones legales, Simulación estocástica (*rollout*) hasta un estado terminal y Retropropagación de recompensas hacia la raíz.

Lo que hace tentador a MCTS para GGP es que no exige heurísticas propias de un juego. Basta con un modelo capaz de dar las acciones legales, aplicar una transición y decir cuándo se terminó la partida; con eso se juegan partidas al azar y los resultados indican qué acciones convienen. El costo es que todo depende de qué tan rápido y bien esté hecho ese modelo, y de cuántas simulaciones alcancen en el tiempo disponible.

3.13. UCT

Una pregunta central en la fase de selección de MCTS es qué rama explorar. Si el agente explora solo la acción que hasta ese momento parece la mejor, corre el riesgo de ignorar alternativas poco visitadas que en realidad serían superiores. Si explora sin criterio, puede perder tiempo de cómputo en acciones de baja calidad. El algoritmo UCT (*Upper Confidence Bounds applied to Trees*) trata este dilema entre explotación y exploración de manera formal [23], [42].

Definición 3.11: Criterio de selección UCT

Para ilustrar UCT en los juegos por turnos del alcance, sea $p(v)$ el rol activo en el nodo v. Por cada simulación que termina en $s_f \in F$, se normalizará el resultado de cada rol como $q_i = G_i(s_f)/100 \in [0, 1]$. En cada nodo visitado se acumulará ese resultado en Q_i y se incrementará su contador N [23], [42].
Si existe un hijo sin visitas, se seleccionará uno de esos hijos antes de evaluar la expresión siguiente; para ilustrar el caso de empate se puede usar cualquier orden fijo. Cuando todos tienen $N(v') > 0$, y por tanto $N(v) > 0$, se aplicará:

$$v^* \in \arg \max_{v' \in \text{hijos}(v)} \left[\frac{Q_{p(v)}(v')}{N(v')} + C\sqrt{\frac{\ln N(v)}{N(v')}} \right], \quad C > 0. \tag{3.33}$$

Aquí $Q_i(v')$ suma las recompensas normalizadas del rol r_i en las simulaciones que visitaron v'. La media se toma desde la perspectiva del jugador activo en el padre, no siempre desde la raíz. Así ambos participantes buscan su propia utilidad. El parámetro C regula la exploración y no se fija como decisión de implementación.

El primer sumando estima la utilidad media y el segundo favorece acciones poco visitadas. Con esta normalización, la elección $C = \sqrt{2}$ reproduce la forma usual de la cota UCB1 [43]; no se traslada sin ajuste a acumulados en la escala $[0, 100]$. No se evalúan cocientes con cero visitas. En estados terminales no hay selección de sucesores; su utilidad se retropropaga.

UCT es un antecedente para la búsqueda por simulación. Sus resultados de consistencia dependen de las hipótesis del análisis, como el horizonte finito o descuento en el artículo original [42]; no aseguran una decisión óptima con un presupuesto finito ni determinan el algoritmo que se implementará.

3.14. Aprendizaje por refuerzo y aprendizaje profundo

El aprendizaje por refuerzo parte de una escena sencilla: un agente en un entorno mira el estado, actúa, recibe una recompensa y cae en otro estado. No hay un maestro que diga en cada paso qué era lo correcto, como ocurre en el aprendizaje supervisado; en un juego, la señal de que algo salió bien o mal puede llegar solo al terminar la partida, y el agente tiene que averiguar qué jugadas la provocaron.

En los juegos, el resultado final (ganar, perder o empatar) puede servir para ajustar la política de decisión del agente. Al combinar esto con redes neuronales profundas se habla de aprendizaje por refuerzo profundo, capaz de aproximar funciones de valor o de política muy complejas,

aunque a cambio pide muchos datos, un entrenamiento intensivo y una buena forma de representar el estado.

Con aprendizaje profundo el problema se complica en GGP. No basta con que el agente aprenda a jugar bien: tiene que hacerlo con juegos que le llegan descritos desde fuera, y eso obliga a que la representación del estado y de las acciones sirva para cualquiera de ellos. Por esa razón lo tratamos aquí como antecedente y como línea de investigación avanzada, y no como algo que este proyecto vaya a implementar.

3.15. AlphaZero como antecedente moderno

AlphaZero es un antecedente importante porque combinó autojuego, aprendizaje por refuerzo profundo y búsqueda Monte Carlo para alcanzar un alto desempeño en Ajedrez, Shogi y Go [7]. Su arquitectura usa una red neuronal que estima una política de acciones y un valor de estado, mientras que MCTS afina la elección de movimientos con búsqueda.

Sin embargo, AlphaZero no resuelve por sí mismo el problema central de este Trabajo Terminal. Aunque usa un algoritmo general para más de un juego, necesita simuladores y representaciones adecuadas para cada dominio. En un GGP el reto es otro: interpretar reglas externas, construir el modelo del juego y lograr que el motor de decisión funcione sin conocimiento específico codificado para cada caso.

Goldwaser y Thielscher llevan ideas de aprendizaje por refuerzo profundo al contexto de GGP y muestran que existe una línea de investigación que intenta unir descripciones formales de juegos con métodos inspirados en AlphaZero [31]. Para el alcance de este trabajo, esas ideas son una referencia del estado del arte y ayudan a acotar el proyecto: se busca una arquitectura general e interpretable, no replicar un sistema de entrenamiento profundo a gran escala.

3.16. Síntesis del marco teórico

Desarrollar un General Game Player exige integrar varias áreas de la inteligencia artificial y de las ciencias computacionales. Primero, una representación formal del juego que separe estados, acciones, transiciones, terminalidad y recompensas. Segundo, un lenguaje de descripción o DSL que exprese las reglas sin codificarlas directamente en el motor. Tercero, un intérprete que convierta esa descripción en una representación interna consultable. Y por último, un agente que elija acciones con algoritmos generales de búsqueda o simulación.

Esa integración se organiza en un flujo desacoplado, que muestra la Figura 3.4. El analizador léxico y sintáctico procesa el archivo de reglas externo y crea en memoria el modelo formal Γ. Sobre este modelo, el generador de estados resuelve consultas abstractas de legalidad, transiciones y condiciones terminales, y con eso el motor de decisión (Minimax, poda alfa-beta o MCTS) elige la mejor acción disponible $\mathbf{a}^*$ sin guardar conocimiento particular del dominio.

Lo que aporta este trabajo no es el mejor jugador de un juego en particular, sino una arquitectura desacoplada: el motor debe seguir igual cuando cambia la descripción del juego. Por eso el Gato sirve como caso funcional completo para validar el flujo de reglas, estados y terminalidad, y el Ajedrez como caso de mayor complejidad, cuyo diseño experimental (posiciones, oponente y métricas) está pendiente de definirse, como se detalla en la Sección 1.6. Con esta delimitación se

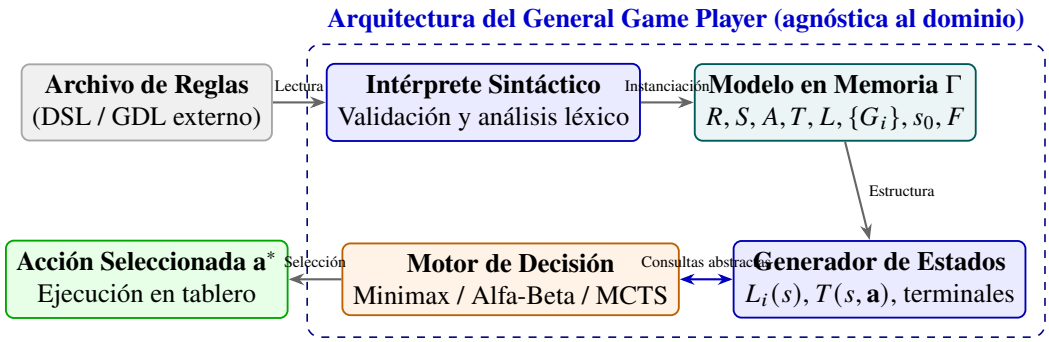


Figura 3.4: Flujo conceptual de ejecución y arquitectura desacoplada del General Game Player. El flujo detallado se presenta en la Figura 5.1.

puede estudiar la generalidad operativa sin confundir el prototipo con un motor especializado de competencia.

Capítulo 4

Análisis

Capítulo 5

Diseño

5.1. Arquitectura conceptual propuesta

La arquitectura de la Figura 5.1 separará reglas, interpretación, representación, generación de sucesores y decisión mediante el modelo formal de la Sección 3.4. El contorno delimitará el núcleo reutilizable; las entradas y la coordinación de la partida quedarán fuera. Se trata de un diseño conceptual, sin fijar tecnología ni acreditar módulos implementados.

1. **Descripción externa.** Cada archivo expresará roles, estado inicial, acciones, legalidad, transiciones, terminalidad y utilidades. La elección del lenguaje requerirá justificar expresividad, restricciones y costo de interpretación.

2. **Intérprete.** Analizará la estructura y las restricciones semánticas acordadas. La descripción aceptada originará el modelo; el rechazo producirá un diagnóstico y detendrá la carga.

3. **Modelo y generador.** El modelo conservará las reglas y la representación de estados; el generador resolverá consultas y construirá sucesores a solicitud del motor. La carga proporcionará s_0, sin materializar por anticipado todo el espacio de Ajedrez.

4. **Motor y ciclo de decisión.** Recibirá estado actual, rol y presupuesto. Si el estado actual es terminal, reportará utilidades y finalizará la partida; en otro caso explorará sucesores y elegirá una acción legal. Los terminales hipotéticos solo servirán para valorar ramas.

La coordinación solicitará al generador la transición real $T(s, \mathbf{a}^*)$ del mismo modelo y recibirá el sucesor para actualizar el turno. Al cargar una descripción aceptada recibirá s_0; en los turnos posteriores utilizará el sucesor recibido. Para formar $\mathbf{a}^*$ se conservará la convención de acciones inactivas. Esta transición real se distinguirá de los sucesores hipotéticos de la búsqueda.

Se estudiarán Minimax con alfa-beta y MCTS/UCT, con heurísticas generales y selección pendiente. El núcleo conservará sus componentes al cambiar el archivo, sin valores de piezas ni procedimientos particulares del dominio. La evaluación de la Sección 1.6.5 registrará contadores y contrastará resultados con referencias externas, sin proporcionar sus respuestas al agente.

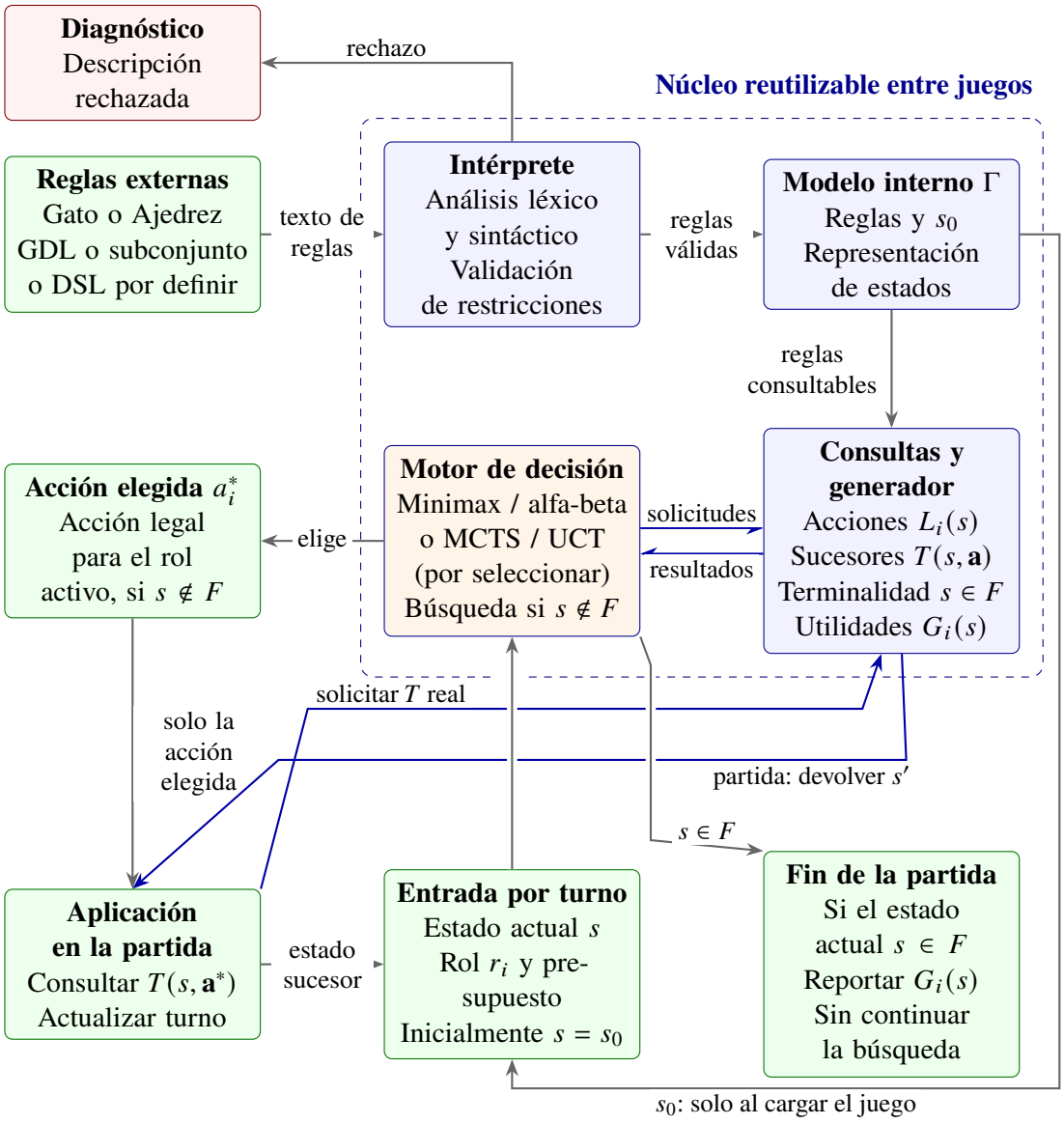


Figura 5.1: Arquitectura propuesta: carga de reglas, consultas de búsqueda y actualización de la partida. Las dos filas principales forman un único flujo; las entradas, salidas y condiciones de terminación se muestran alrededor del núcleo.

5.2. Prototipos de interfaz aprobados

Las Figuras 5.2–5.6 documentan prototipos visuales aprobados para comunicar cómo podría organizarse la interacción con el agente. Retoman los límites de responsabilidad de la arquitectura de la Figura 5.1: la interfaz recibe una descripción externa, muestra el estado de una partida y presenta la preparación o el reporte de una evaluación. No describen una interfaz implementada ni fijan una tecnología, la sintaxis definitiva de las reglas o el algoritmo de decisión. Los controles, estados y valores visibles son ilustrativos y no constituyen evidencia de ejecución.

Carga y validación de reglas. La Figura 5.2 ilustra la entrada de una descripción externa mediante un archivo de ejemplo de Gato. La vista semántica hace explícitos los roles, el estado inicial, las acciones y la condición de fin; el panel lateral representa el resultado que se esperaría de validar roles, transición, terminalidad y utilidades. La leyenda "descripción aceptada" es un estado propuesto de la interfaz, no una afirmación de que exista ya un intérprete ni de que se haya elegido GDL, un subconjunto de GDL o un DSL propio.

Partida de Gato. La Figura 5.3 propone una vista para una partida entre dos agentes. Presenta el tablero, el turno activo, las acciones legales, un historial y el espacio reservado para la siguiente decisión. Con ello separa la visualización de la partida de las consultas que el motor deberá resolver sobre legalidad y transición. La posición pausada y sus acciones se muestran solo como ejemplo de interacción, sin acreditar que se generen a partir de reglas ejecutables.

Partida de Ajedrez. La Figura 5.4 reutiliza la misma organización para Ajedrez: tablero, turno, acciones legales, historial y próxima decisión. Su propósito es hacer visible que la interfaz no queda ligada a la geometría ni a las piezas de un único dominio; el cambio de juego corresponderá a la descripción externa y al modelo interno. La posición inicial y las veinte acciones mostradas son datos visuales de ejemplo, no una prueba de que el agente ya soporte las reglas completas de Ajedrez.

Preparación de la evaluación. La Figura 5.5 representa la configuración previa de una comprobación de primera jugada en Ajedrez. Distingue el banco de casos, las condiciones de ejecución y el criterio de comparación, de acuerdo con la propuesta de métricas de la Sección 1.6.5. Las 30 posiciones, una ejecución por posición y el presupuesto de 1000 ms son valores de ejemplo; el tamaño definitivo del banco, las repeticiones, el presupuesto y el protocolo de Ajedrez continúan sujetos a acuerdo. La indicación de que las respuestas pertenecen al evaluador preserva la separación entre referencia independiente y agente evaluado.

Resultados de evaluación. La Figura 5.6 propone cómo reportar los indicadores definidos para la validación: acierto, tiempo por decisión, nodos expandidos, posiciones distintas, ejecuciones y detalle por caso. También conserva el motivo de terminación para que los tiempos agotados y otras incidencias permanezcan visibles en el registro. Los valores numéricos, los movimientos y el resumen de 4 de 5 casos son datos simulados; la figura no informa resultados experimentales ni selecciona un algoritmo de búsqueda.



Figura 5.2: Prototipo aprobado de carga y revisión de una descripción externa de Gato. La validación y el contenido del archivo son ilustrativos; la sintaxis del lenguaje permanece por definir.



Figura 5.3: Prototipo aprobado de una partida de Gato entre agentes. El tablero, historial, acciones legales y próxima decisión representan un estado ilustrativo.



Figura 5.4: Prototipo aprobado de una partida de Ajedrez entre agentes. La posición y las acciones son ilustrativas.

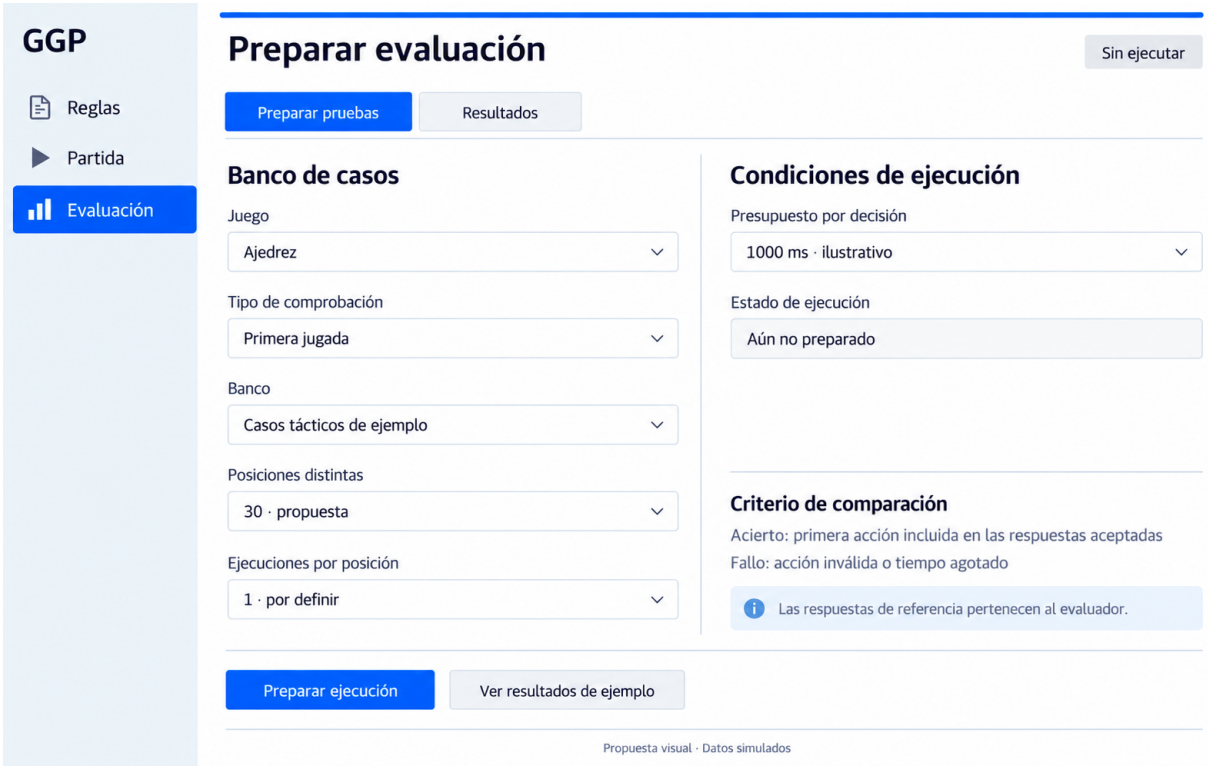


Figura 5.5: Prototipo aprobado de preparación de una evaluación táctica de primera jugada en Ajedrez. Las cantidades, el presupuesto y el estado de ejecución son ilustrativos y no describen un experimento realizado.



Figura 5.6: Prototipo aprobado del reporte de resultados de una evaluación. Las métricas y el detalle por caso muestran datos simulados, sin atribuir una ejecución real ni desempeño al agente.

Capítulo 6

Desarrollo

Capítulo 7

Pruebas

Capítulo 8

Conclusiones y trabajo a futuro

Referencias

[1] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 3.ª ed. Prentice Hall, 2010.

[2] A. L. Samuel, «Some Studies in Machine Learning Using the Game of Checkers,» *IBM Journal of Research and Development*, vol. 3, n.º 3, págs. 210-229, 1959. DOI: 10.1147/rd.33.0210

[3] A. M. Turing, «Computing Machinery and Intelligence,» *Mind*, vol. 59, n.º 236, págs. 433-460, 1950. DOI: 10.1093/mind/LIX.236.433

[4] M. Campbell, A. J. Hoane y F.-h. Hsu, «Deep Blue,» *Artificial Intelligence*, vol. 134, n.º 1–2, págs. 57-83, 2002. DOI: 10.1016/S0004-3702(01)00129-1

[5] J. Schaeffer, N. Burch, Y. Björnsson, A. Kishimoto, M. Müller, R. Lake, P. Lu y S. Sutphen, «Checkers Is Solved,» *Science*, vol. 317, n.º 5844, págs. 1518-1522, 2007. DOI: 10.1126/science.1144079

[6] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel y D. Hassabis, «Mastering the Game of Go with Deep Neural Networks and Tree Search,» *Nature*, vol. 529, n.º 7587, págs. 484-489, 2016. DOI: 10.1038/nature16961

[7] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan y D. Hassabis, «A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go Through Self-Play,» *Science*, vol. 362, n.º 6419, págs. 1140-1144, 2018. DOI: 10.1126/science.aar6404

[8] M. Genesereth, N. Love y B. Pell, «General Game Playing: Overview of the AAAI Competition,» *AI Magazine*, vol. 26, n.º 2, págs. 62-72, 2005. dirección: https://logic.stanford.edu/publications/genesereth/aaai.pdf

[9] M. Genesereth y M. Thielscher, *General Game Playing*. Morgan & Claypool Publishers, 2014. dirección: https://logic.stanford.edu/ggp/chapters/cover.html

[10] N. Love, T. Hinrichs, D. Haley, E. Schkufza y M. Genesereth, «General Game Playing: Game Description Language Specification,» Stanford Logic Group, inf. téc. LG-2006-01, 2008. dirección: https://logic.stanford.edu/ggp/

[11] Google. «The N-queens Problem,» visitado 25 de oct. de 2026. dirección: https://developers.google.com/optimization/cp/queens

[12] C. E. Shannon, «Programming a Computer for Playing Chess,» *Philosophical Magazine*, vol. 41, n.º 314, págs. 256-275, 1950. DOI: 10.1080/14786445008521796

[13] The Stockfish developers. «Stockfish: A free and strong UCI chess engine,» visitado 25 de oct. de 2026. dirección: https://github.com/official-stockfish/Stockfish

[14] M. Schmid, M. Moravcik, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, G. Z. Holland, E. Davoodi, A. Christianson y M. Bowling, «Student of Games: A unified learning algorithm for both perfect and imperfect information games,» *Science Advances*, vol. 9, n.º 46, eadg3256, 2023. DOI: 10.1126/sciadv.adg3256

[15] E. Sánchez Catarino y G. Linares Villegas, «Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta,» Protocolo de Trabajo Terminal 2027-A178, Escuela Superior de Cómputo, Instituto Politécnico Nacional. Documento de trabajo proporcionado por los autores.

[16] N. Fiekas. «Core — python-chess documentation,» visitado 17 de oct. de 2026. dirección: https://python-chess.readthedocs.io/en/latest/core.html

[17] Lichess. «Lichess open database: Puzzles,» visitado 17 de oct. de 2026. dirección: https://database.lichess.org/#puzzles

[18] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap y D. Silver, «Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model,» *Nature*, vol. 588, n.º 7839, págs. 604-609, 2020. DOI: 10.1038/s41586-020-03051-4

[19] J. Clune, «Heuristic Evaluation Methods for General Game Playing,» en *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, 2007, págs. 1133-1139.

[20] S. Schiffel y M. Thielscher, «Fluxplayer: A Successful General Game Player,» en *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, 2007, págs. 1191-1196.

[21] H. Finnsson e Y. Björnsson, «Simulation-Based Approach to General Game Playing,» en *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, 2008, págs. 259-264.

[22] H. Finnsson, «Generalized Monte-Carlo Tree Search Extensions for General Game Playing,» en *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, 2012, págs. 1550-1556.

[23] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis y S. Colton, «A Survey of Monte Carlo Tree Search Methods,» *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, n.º 1, págs. 1-43, 2012. DOI: 10.1109/TCIAIG.2012.2186810

[24] S. Draper y A. Rose. «Sancho,» visitado 25 de oct. de 2026. dirección: https://sanchoggp.github.io/sancho-ggp/

[25] S. Gelly y D. Silver, «Monte-Carlo tree search and rapid action value estimation in computer Go,» *Artificial Intelligence*, vol. 175, n.º 11, págs. 1856-1875, 2011. DOI: 10.1016/j.artint.2011.03.007

[26] M. Thielscher, «A General Game Description Language for Incomplete Information and Stochastic Games,» en *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, 2010, págs. 994-999. DOI: 10.1609/aaai.v24i1.7677

[27] É. Piette, D. J. N. J. Soemers, M. Stephenson, C. F. Sironi, M. H. M. Winands y C. Browne, «Ludii – The Ludemic General Game System,» en *ECAI 2020*, 2020, págs. 411-418. DOI: 10.3233/FAIA200120 dirección: https://arxiv.org/abs/1905.05013

[28] D. J. N. J. Soemers, E. Piette, M. Stephenson y C. Browne, «The Ludii Game Description Language is Universal,» en *Proceedings of the 2024 IEEE Conference on Games (CoG)*, 2024, págs. 1-8. DOI: 10.1109/CoG60054.2024.10645550

[29] J. Kowalski, M. Mika, J. Sutowicz y M. Szykuła, «Regular Boardgames,» en *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence*, 2019, págs. 1699-1706. DOI: 10.1609/aaai.v33i01.33011699

[30] M. Lanctot, E. Lockhart, J.-B. Lespiau, V. Zambaldi, S. Upadhyay, J. Pérolat, S. Srinivasan, F. Timbers, K. Tuyls, S. Omidshafiei, D. Hennes, D. Morrill, P. Feng, T. Biro, J. Hou, F. Bazzana, M. Schmid, D. Geiger, S. Liu, J. Schrittwieser, I. Antonoglou, T. Graepel y M. Bowling, «OpenSpiel: A Framework for Reinforcement Learning in Games,» *arXiv preprint arXiv:1908.09453*, 2019. DOI: 10.48550/arXiv.1908.09453

[31] A. Goldwaser y M. Thielscher, «Deep Reinforcement Learning for General Game Playing,» en *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020, págs. 1701-1708. DOI: 10.1609/aaai.v34i02.5533

[32] H. W. Kuhn, «Extensive Games and the Problem of Information,» en *Contributions to the Theory of Games (AM-28), Volume II*, Princeton University Press, 1953, págs. 193-216. DOI: 10.1515/9781400881970-012

[33] M. J. Osborne y A. Rubinstein, *A Course in Game Theory*. Cambridge, MA: MIT Press, 1994.

[34] J. McCarthy y P. J. Hayes, «Some Philosophical Problems from the Standpoint of Artificial Intelligence,» en *Machine Intelligence 4*, B. Meltzer y D. Michie, eds., Edinburgh University Press, 1969, págs. 463-502.

[35] R. Reiter, «On Closed World Data Bases,» en *Logic and Data Bases*, H. Gallaire y J. Minker, eds., Plenum Press, 1978, págs. 55-76. DOI: 10.1007/978-1-4684-3384-5_3

[36] B. Bonet y H. Geffner, «Planning as Heuristic Search,» *Artificial Intelligence*, vol. 129, n.º 1–2, págs. 5-33, 2001. DOI: 10.1016/S0004-3702(01)00108-4

[37] J. Hoffmann y B. Nebel, «The FF Planning System: Fast Plan Generation Through Heuristic Search,» *Journal of Artificial Intelligence Research*, vol. 14, págs. 253-302, 2001. DOI: 10.1613/jair.855 dirección: https://arxiv.org/abs/1106.0675

[38] A. Cropper, R. Evans y M. Law, «Inductive General Game Playing,» *Machine Learning*, vol. 109, págs. 1393-1434, 2020. DOI: 10.1007/s10994-019-05843-w

[39] J. von Neumann, «Zur Theorie der Gesellschaftsspiele,» *Mathematische Annalen*, vol. 100, n.º 1, págs. 295-320, 1928. DOI: 10.1007/BF01448847

[40] D. E. Knuth y R. W. Moore, «An Analysis of Alpha-Beta Pruning,» *Artificial Intelligence*, vol. 6, n.º 4, págs. 293-326, 1975. DOI: 10.1016/0004-3702(75)90019-3

[41] R. Coulom, «Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search,» en *Computers and Games (CG 2006)*, ép. Lecture Notes in Computer Science, vol. 4630, Springer, 2007, págs. 72-83. DOI: 10.1007/978-3-540-75538-8_7

[42] L. Kocsis y C. Szepesvári, «Bandit Based Monte-Carlo Planning,» en *Machine Learning: ECML 2006*, ép. Lecture Notes in Computer Science, vol. 4212, Springer, 2006, págs. 282-293. DOI: 10.1007/11871842_29

[43] P. Auer, N. Cesa-Bianchi y P. Fischer, «Finite-time Analysis of the Multiarmed Bandit Problem,» *Machine Learning*, vol. 47, n.º 2–3, págs. 235-256, 2002. DOI: 10.1023/A:1013689704352

Anexos

Cronogramas

Los siguientes cronogramas reproducen las actividades, los responsables y los periodos del protocolo del TT 2027-A178 [15]. Las celdas sombreadas indican el periodo programado; no representan actividades concluidas ni porcentajes de avance. Se conservan los meses de agosto a junio sin asignar años o fechas adicionales.

Sánchez Catarino Edgar

Actividad	Ago	Sep	Oct	Nov	Dic	Ene	Feb	Mar	Abr	May	Jun
Fundamentación teórica del motor y representación matemática	■	■	■								
Programación de reglas, validación y generación de movimientos				■	■	■					
Implementación del agente inteligente y algoritmos de optimización de búsqueda.							■	■			
Integración del motor base							■	■			
Desarrollo del archivo de descripción del juego ajedrez									■		
Redacción de los capítulos técnicos y manuales del documento de Trabajo Terminal.									■	■	
Revisión final del documento y preparación del material para la defensa.											■

Linares Villegas Gustavo

Actividad	Ago	Sep	Oct	Nov	Dic	Ene	Feb	Mar	Abr	May	Jun
Investigación del estado del arte en agentes lúdicos y definición del problema.	■	■	■								
Diseño algorítmico de los árboles de búsqueda y propuesta de la función heurística.			■	■	■	■					
Diseño y ejecución de experimentos para evaluar el rendimiento algorítmico.							■	■			
Análisis estadístico de resultados y redacción de conclusiones en el Trabajo Terminal.									■		
Redacción de los capítulos técnicos y manuales del documento de Trabajo Terminal.									■	■	
Revisión final del documento y preparación del material para la defensa.											■

Dependencia por revisar. Los experimentos están programados para febrero y marzo, mientras que el archivo de descripción de Ajedrez está programado para abril. Antes de ejecutar las pruebas, se deberá revisar la disponibilidad de ese archivo y delimitar los experimentos que dependan de él. Se conserva el calendario original y no se supone que las actividades ya hayan sido realizadas.